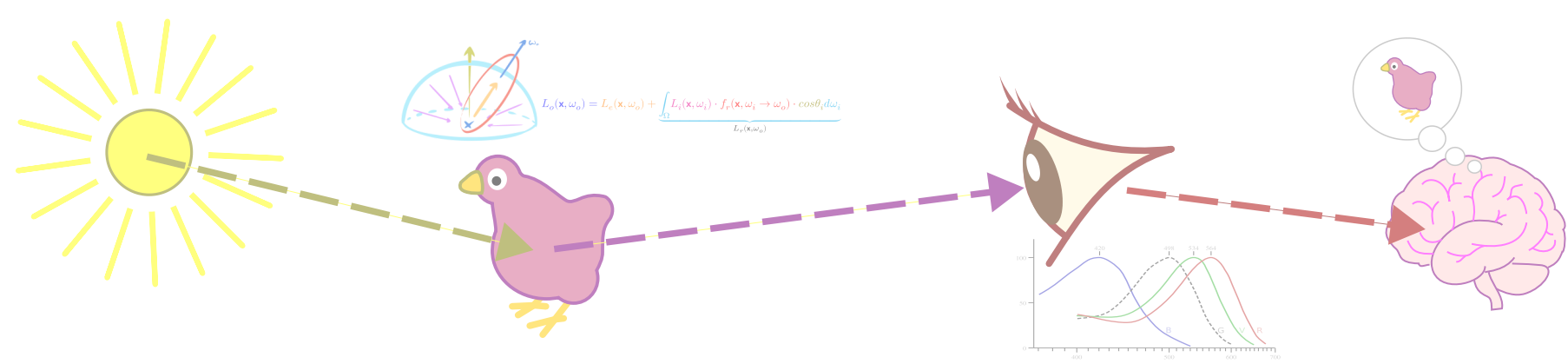
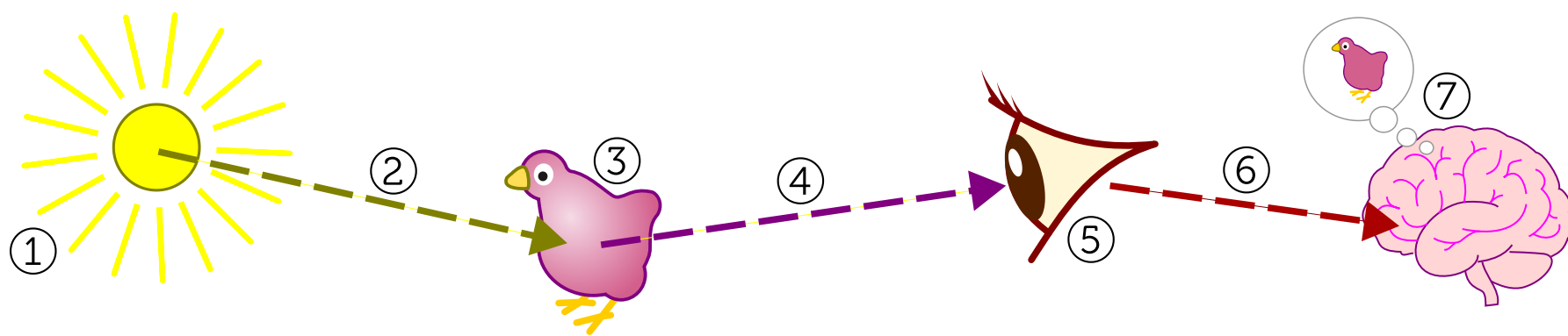


Color e Iluminación



¿CÓMO/POR QUÉ VEMOS?

1. Una fuente de luz emite "luz" (ej: el sol)
2. Esa viaja hasta chocar contra los objetos
3. **En cada choque, "parte" se absorbe, "parte" rebota**
4. "Parte" de lo que rebota, viajará en dirección al ojo
5. **El ojo tiene sensores que captan la luz**
6. Los sensores envían una señal al cerebro
7. El cerebro interpreta esa información



Esta es una versión rápida y poco precisa de todo lo involucrado en la visión. Es para tener una referencia global e identificar los pasos que nos interesa modelar.

Vemos las cosas gracias a la luz, que sale de una fuente, rebota contra los objetos, y llega a nuestros ojos que son los sensores que la detectan. Si vemos un objeto, por ejemplo, verde, es porque el objeto recibió luz (generalmente blanca, que sería la combinación de todos los colores), y absorbió todas las componentes de la luz, menos el verde; La "parte verde" de la luz es la que rebotó y llegó a nuestros ojos.

Deberíamos simular los pasos 1 a 4 para generar una imagen virtual, pero nos centraremos mayormente en el paso 3, que será el más determinante.

No modelaremos la fuente de luz (1) con mucho detalle. En general la consideraremos simplemente un punto en la escena que emite luz blanca en todas las direcciones. Aunque las luces de la realidad no son puntuales, y además podríamos querer plantear algo como una lámpara o linterna, que tiene una "dirección" preferencial alrededor de la cual emite luz.

Omitiremos en la mayoría de los casos los pasos 2 y 3. Pero podría haber "algo" en el ambiente que haga que la luz se altere en ese camino. Para empezar, debería atenuarse con la distancia. Pero además, podríamos querer modelar niebla, humo, tierra suspendida, estar bajo el agua, etc... algo que influya en estos pasos.

El paso 3 será el más determinante porque será el que nos permita modelar el color y las propiedades del material de un objeto (por ejemplo, si es rugoso, o pulido.. no es lo mismo una tela verde que un plástico verde).

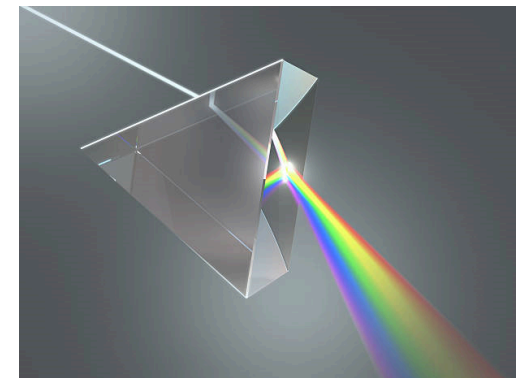
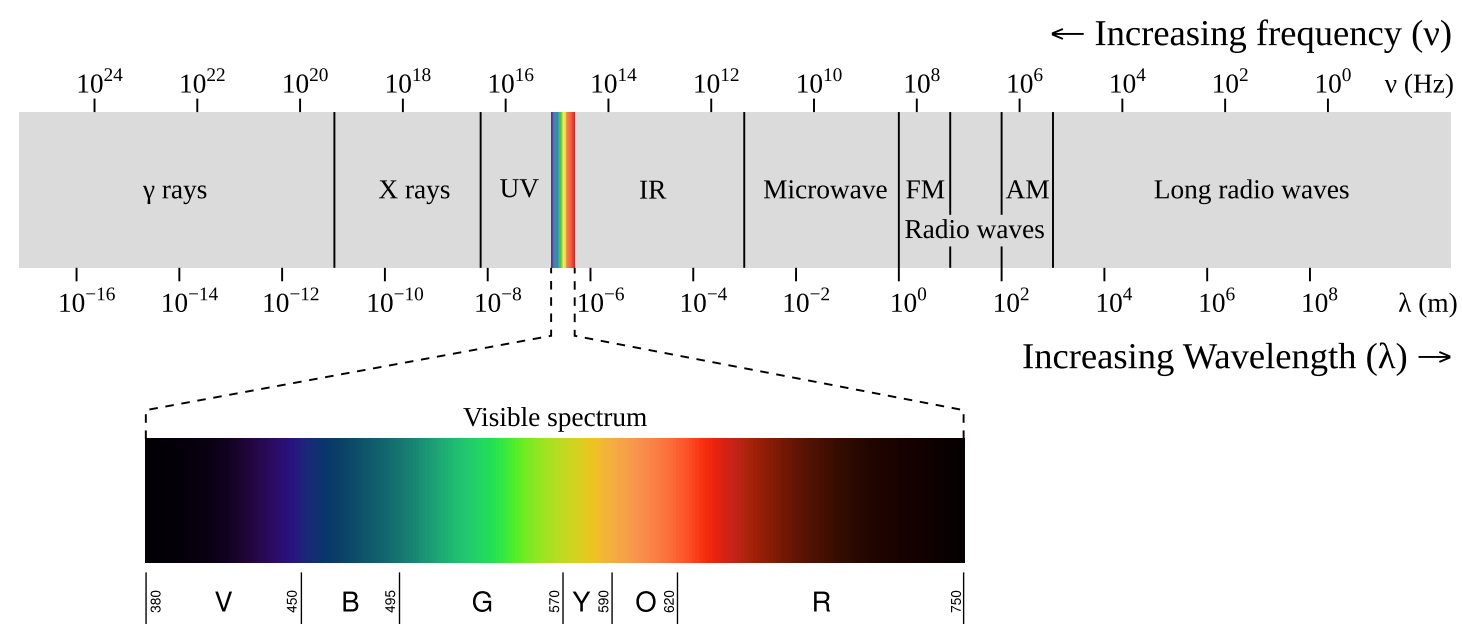
También nos detendremos en el punto 5, a hablar brevemente sobre cómo funciona el ojo humano, ya que eso explica por qué es como es la tecnología que usamos para reproducir las imágenes (los monitores e impresoras, por ejemplo).

Pero el camino no termina ahí. La información censada por los ojos se transforma camino al cerebro (6), y esa versión transformada de la señal original es la que luego el cerebro debe interpretar (7). Estos procesos son muy complejos y escapan a nuestra materia y nuestro tiempo disponible. Nos limitaremos a intentar reproducir, con ciertos errores aceptables, el estímulo para el ojo, y dejaremos de lado todo lo que ocurre después.

Parte 1: Color

En la primera mitad, hablaremos de qué es un color y cómo modelarlo. Para ello será importante repasar algo de la física de la luz, y de la fisionomía del ojo humano. No lo explicaremos ni en profundidad ni con toda la rigurosidad que se podría, sino solo lo suficiente como para justificar o entender algunos aspectos de la tecnología actual.

¿QUÉ ES UN COLOR?



Se le llama "radiación electromagnética" a las ondas generadas por el campo electromagnético. Ondas que se propagan por el espacio a la velocidad de la luz transportando energía. Como toda onda, se caracteriza por una frecuencia o una longitud de onda (son inversas una de otra).

El diagrama muestra el "espectro electromagnético", una especie de catálogo que señala cómo denominamos a esta radiación en distintos rangos del espectro. En algún punto en medio (entre las long. de onda de 400nm a 700nm) está la parte "visible", que es lo que comúnmente llamamos "luz".

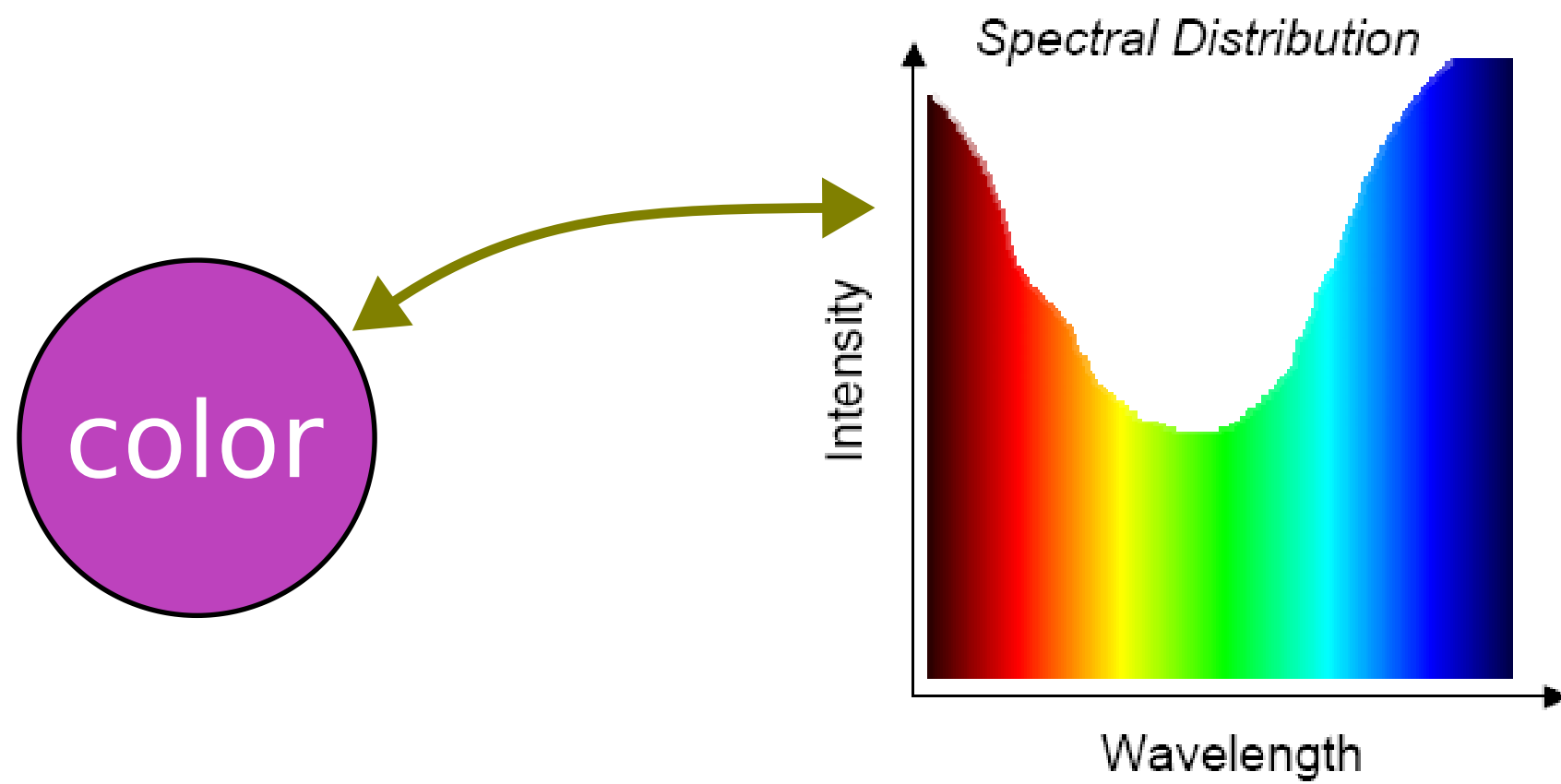
¿Por qué esa parte es visible? Simplemente porque tenemos sensores en los ojos que reaccionan (células que se excitan y mandan señales al cerebro) cuando reciben radiación de esas longitudes de onda, y no de otras. Para otros animales o seres vivos, el rango visible podría ser levemente diferente si sus "sensores" (sus ojos) son diferentes.

Cada longitud de onda en ese rango visible se asocia con un color. Se puede apreciar cómo la luz blanca o la luz del sol se descomponen en estos colores gracias a la refracción (cuando pasan por un prisma, o por las gotas de agua suspendidas luego de la lluvia y dan lugar al arcoíris).

Pero... Dije "cada longitud de onda en ese rango visible se asocia con un color". ¿Dónde están el marrón o el magenta en el arcoíris? Lo contrario, "cada color corresponde con una longitud de onda", no es cierto.

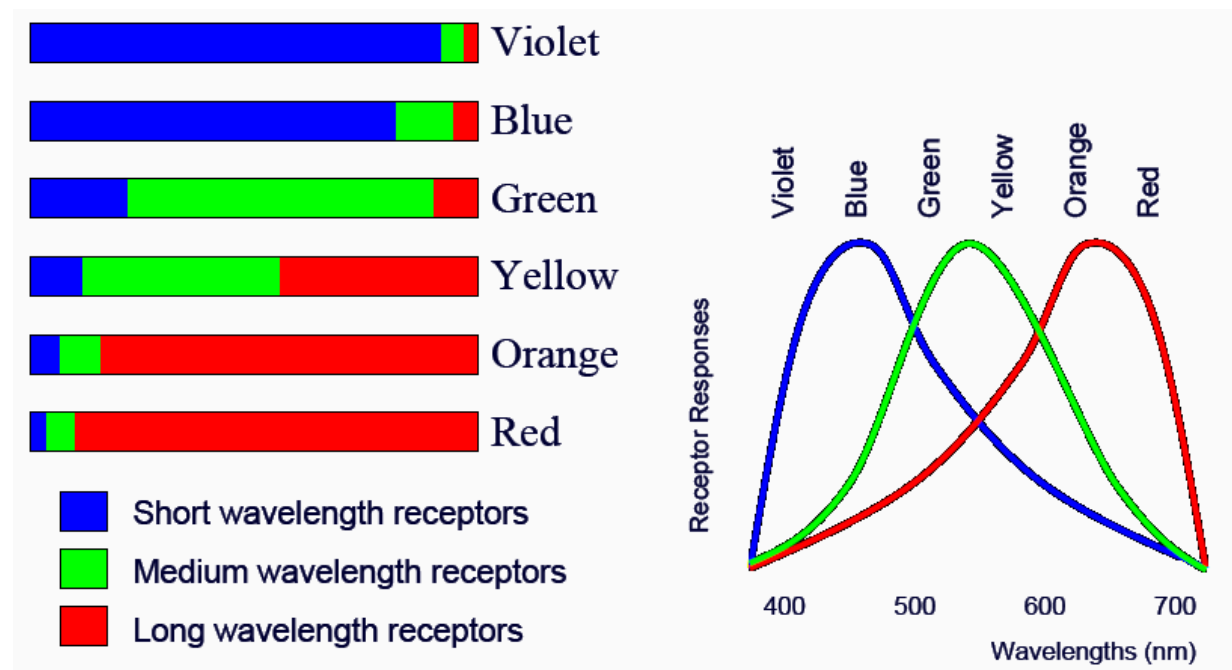
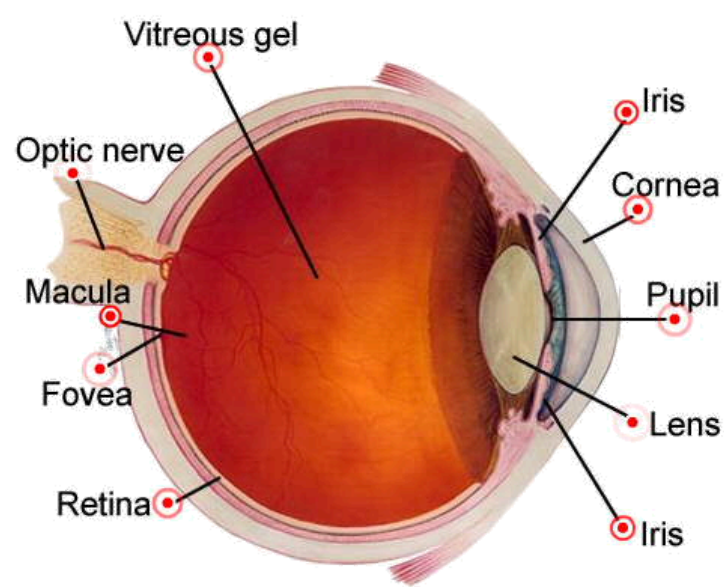
¿QUÉ ES UN COLOR?

Un color es una distribución o espectro de longitudes de onda electromagnéticas (dentro del rango visible)



Un color es una distribución, función o espectro de longitudes de onda electromagnéticas. Es decir, un color generalmente está formado por contribuciones de diferentes longitudes de onda. La función o distribución nos dice cuánta energía para cada frecuencia o longitud de onda del rango visible (que son infinitas, la frecuencia es un valor real, continuo).

EL OJO HUMANO



► Bastones:

- Visión B/N con poca luz
- Visión periférica

► Conos:

- Responsables del color
- 3 tipos: approx R G B

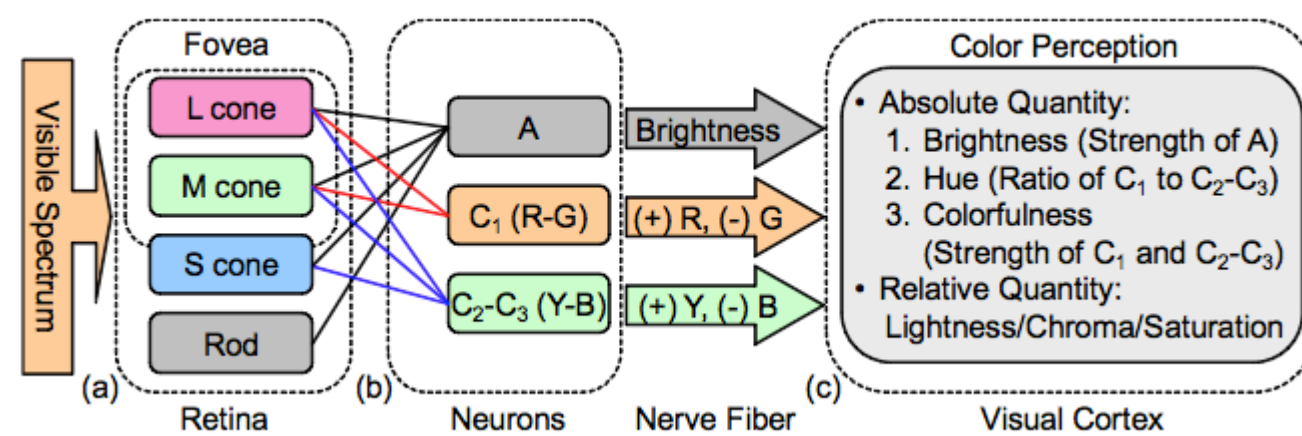
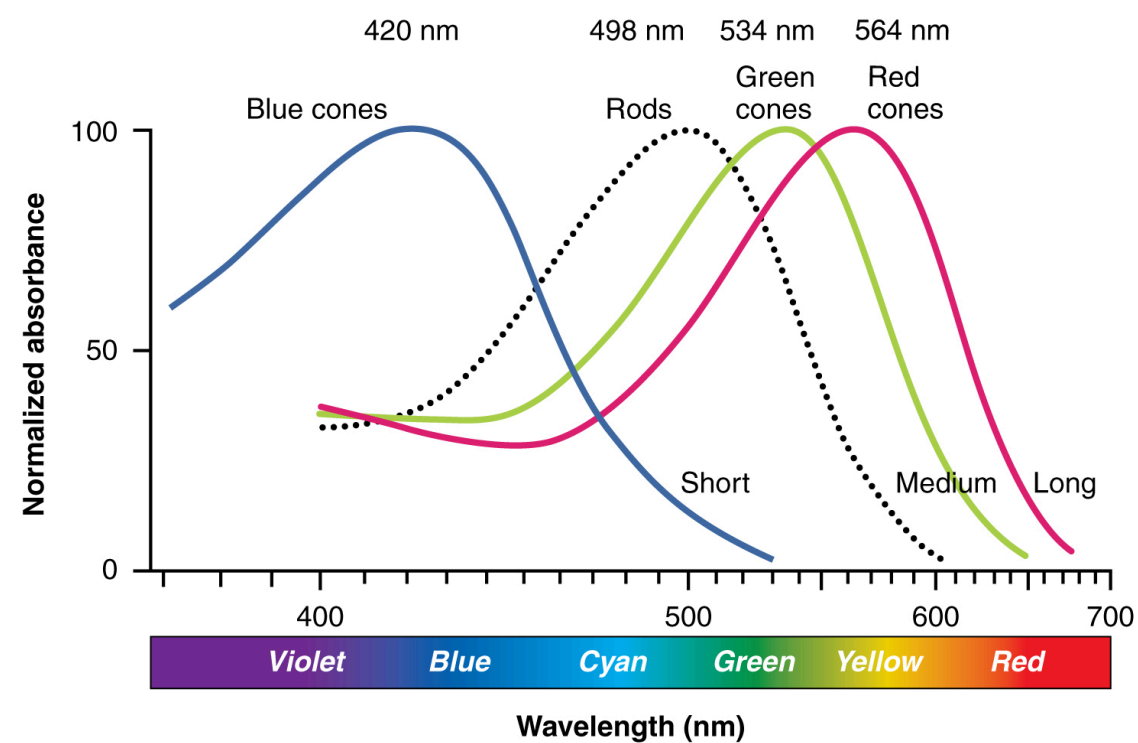
La luz ingresa al ojo a través de la pupila y llega a las células del fondo, alrededor de la fovea, que son los llamados conos y bastones. Estos son los verdaderos sensores fotosensibles. En la parte central de la "imagen" que vemos, predominan los conos; en la periferia los bastones.

Los bastones requieren poco estímulo de luz para enviar una señal al cerebro, por eso son los que nos permiten al menos distinguir formas en condiciones de muy poca iluminación. Pero son los conos los encargados de dar color y detalle a la imagen. Los conos necesitan mucha luz para activarse (por eso la pupila se puede dilatar para dejar ingresar más luz), y permiten captar el color porque hay distintos tipos de conos que resultan sensibles a distintas longitudes de onda (no así los bastones, que son todos "iguales").

Para cada tipo de cono, podemos graficar su respuesta en función de la frecuencia. Simplificando mucho, se suele decir que hay 3 tipos de conos, cuyas respuestas tienen sus picos en diferentes zonas de frecuencia que asociamos a los colores rojo, verde y azul. De ahí el RGB de los monitores.

El corolario de esto es que nuestros ojos distinguen los colores de acuerdo a lo que captan los 3 tipos de conos. Dos colores podrían estar compuestos por diferentes distribuciones de longitudes de onda, pero generar el mismo estímulo para el ojo, y entonces los percibiríamos igual. Digamos que se puede generar una señal de "rojo" para el cerebro con solo un poco de energía en las frecuencias donde los conos "rojos" son más sensible; o generar la misma señal con más energía en las frecuencias donde los conos "rojos" son solo "un poco" sensibles. La señal que estos "sensores" generan sería como la integral del "producto" entre ambas funciones (el color o la distribución intensidad de energía de la luz que llega, y la de sensibilidad del tipo de cono, ambas funciones de la longitud de onda).

EL OJO HUMANO



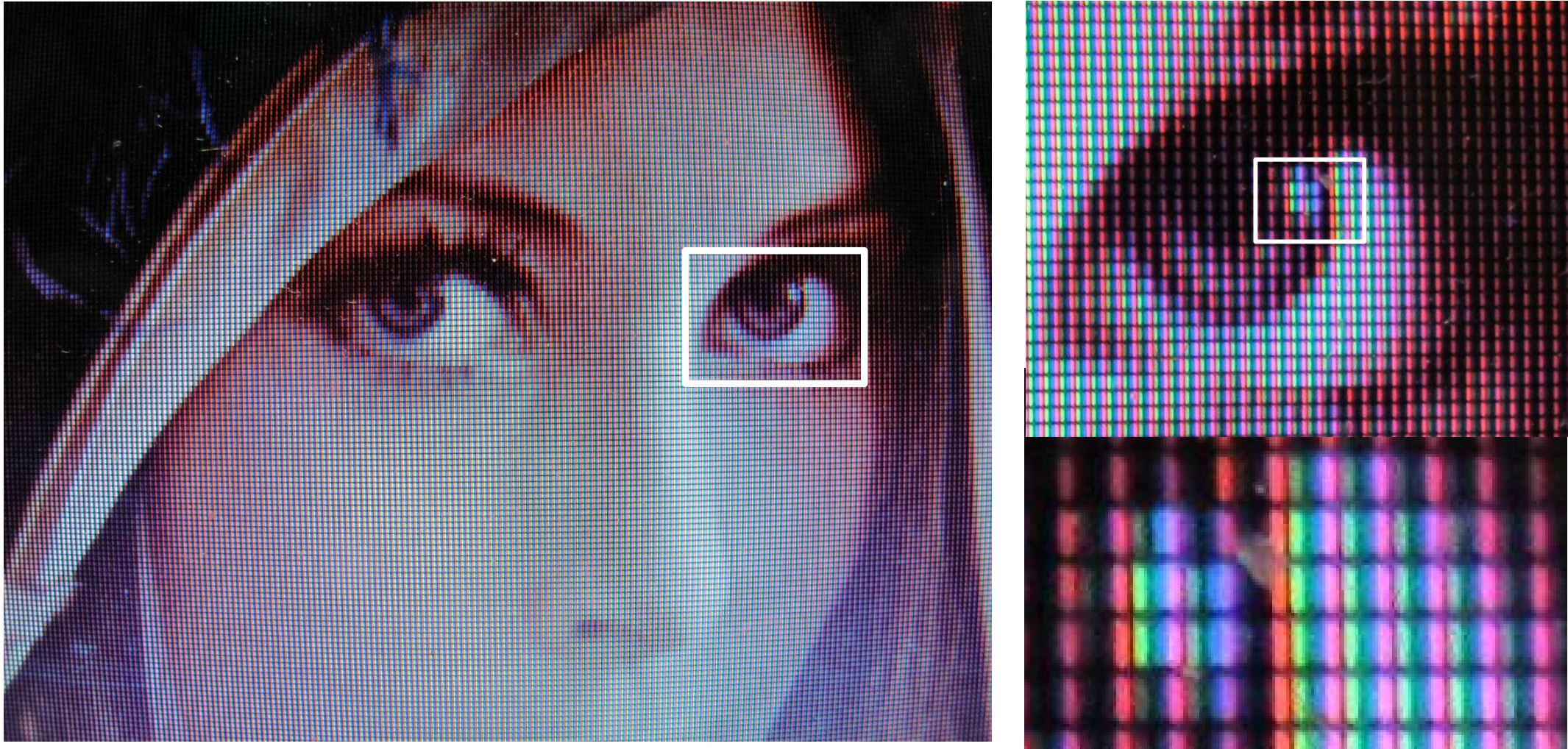
En la realidad las tres "montañitas" de la gráfica de sensibilidad de los tres tipos de conos nos son tan simples. No están tan separadas entre sí (en particular verde y rojo están muy cerca, por eso el tipo de daltonismo más común confunde estos tonos). Sus picos no coinciden con rojo, verde y azul (pero rojo, verde y azul sí son frecuencias que apuntan a excitar un tipo de cono en particular). La altura de las 3 curvas no sería la misma, pero tampoco hay igual cantidad de conos de cada tipo en el ojo... etc, etc.

Luego está el camino al cerebro. El cerebro no toma por entrada tres niveles de rojo, verde y azul (y eventualmente un cuarto que sería el de los bastones), sino que estos canales se combinan formando otros 3 nuevos canales que serán la verdadera entrada de la corteza visual. Es como si dos sensores me dan dos señales, por ejemplo: 5 y 3. Pero yo las transformo en dos nuevos canales: 8 y 2, que son uno la suma y el otro la resta. Y envío a la etapa de procesamiento esos nuevos valores, (8 y 2) en lugar de los originales (3 y 5). La información original está, pero codificada/transformada, y no directamente como se censó. En el caso de los colores, hay una canal por ejemplo que se forma contraponiendo (digamos que restando) rojo y verde (y por eso no podemos ver un "rojo verdoso").

Finalmente el cerebro debe interpretar. Habrán visto muchas veces imágenes donde dos cosas que parecen de tonos muy diferentes son en realidad exactamente el mismo color, que parece que se mueven pero están quietas, que si las miramos fijas y luego miramos para otro lado producen algún efecto, etc. Todo esto tiene que ver con cómo procesa el cerebro esa información, el contexto espacial y temporal de la misma, etc.

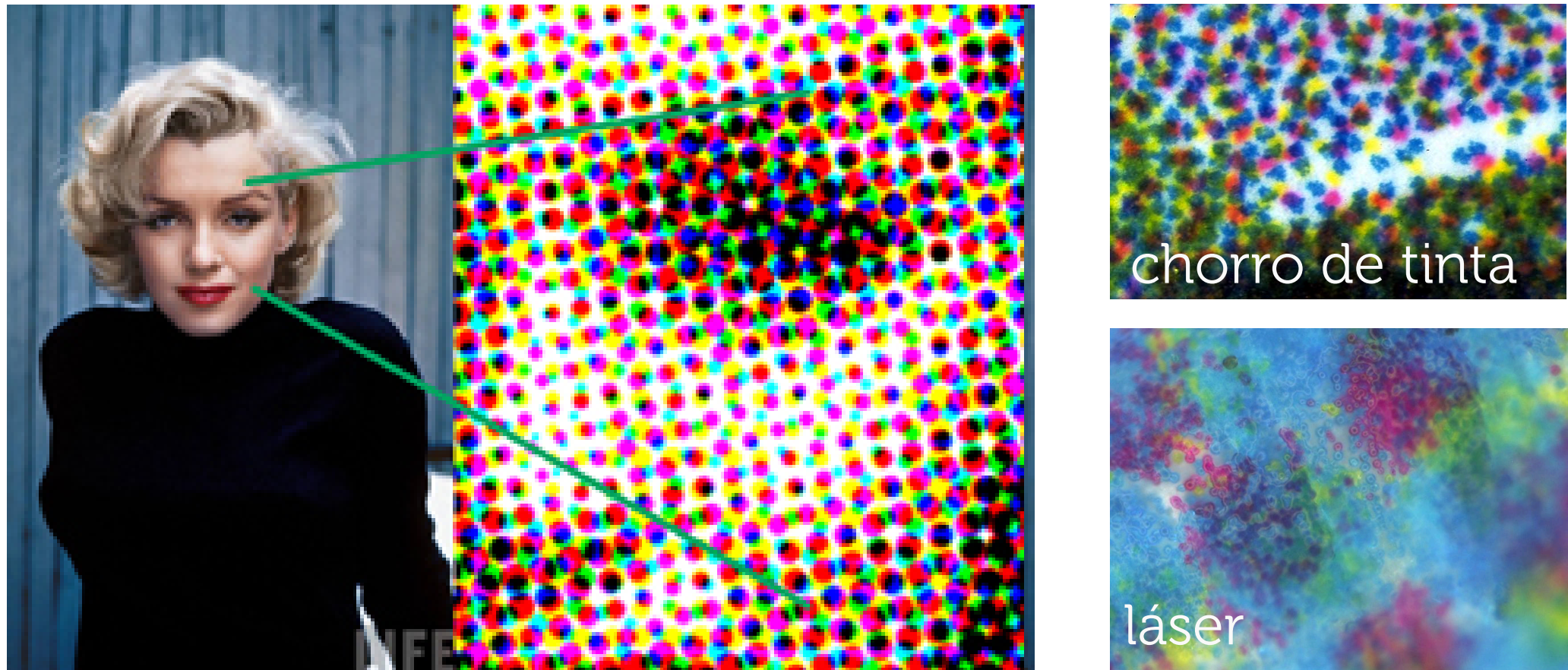
Nada de esto último llegaremos a tener en cuenta. Solo lo mencionamos para dar una idea de que hay muchísimo más para hablar sobre la percepción del color y las imágenes, y que la realidad es mucho más complicada. La idea básica nos alcanza para definir qué es realmente un color, y por qué utilizamos RGB para modelarlo, y eso es lo más importante de esta parte.

REPRODUCCIÓN DEL COLOR: MONITOR



Esta una foto con mucho zoom de un monitor RGB. Se puede ver cómo cada "pixel" está en realidad formado por tres "lucecitas" (sub-píxeles) rojas, verdes y azules. No todos los monitores tienen los subpíxeles distribuidos de la misma forma, pero sí usan siempre rojo, verde y azul.

REPRODUCCIÓN DEL COLOR: IMPRESORA

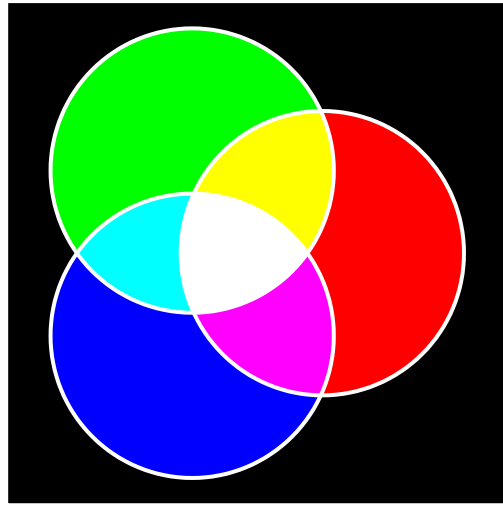


Pero en el caso de las impresoras, los colores "primarios" que usan (las tintas) son cian, magenta y amarillo. A continuación explicamos por qué.

Notar también en esta imagen, en las fotos de los dos tipos de impresora, cómo los "puntos" cian, magenta y amarillo con los que las impresoras generan los colores están menos definidos que en un monitor. En el monitor los 3 subpíxeles están perfectamente alineados y se controlan por separado. En la impresora, esta combinación es imperfecta, por las tintas, y principalmente por el papel. El monitor controla su propio "papel", mientras que la impresora debe trabajar con lo que ponga el usuario (que no será generalmente papel fotográfico). De ahí que la mezcla de colores es muy precisa en el monitor, y muy poco en la impresión.

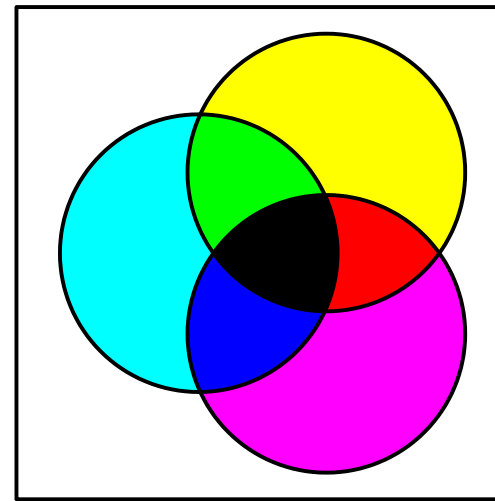
COLORES ADITIVOS Y SUSTRATIVOS

Aditivos



- Bases: R G B ¿A?
- Parten del Negro (monitor)
- Agregan Color/Luz (leds)
- Todo al máximo forma blanco

Sustractivos



- Bases: C M Y ¿K?
- Parten del blanco (papel)
- Absorben Color/Luz (tintas)
- Todo al máximo forma negro

RGB[A]

Entonces, la explicación del ojo deja claro por qué R, G, B. Cada led busca generar luz que excite un tipo de cono diferente, para combinarlos y generar la sensación de los diferentes colores. A este modelo se lo llama aditivo, porque parte del negro (no hay luz), y debe "agregar luz" (emitir) para formar los colores.

Va a ser muy común anexar un cuarto canal llamado "alfa" que usualmente se usa para opacidad ($\alpha=1$ es para objetos opacos, $\alpha=0$ para transparentes). Esto ya no tiene nada que ver con el ojo, alfa en realidad es un canal extra que podría usarse para cualquier otra cosa, pero lo más habitual es hablar de colores RGBA y donde "por defecto" α es "opacidad" (aunque se le llama erróneamente "transparencia", que es justo lo contrario).

CMY[K]

En el caso de la impresora, el modelo es sustractivo. La hoja es blanca, lo que quiere decir que refleja hacia el ojo todos los colores. Si quiero que se vea verde, debo poner tinta que absorba todo menos el verde. La tinta cian absorbe rojo, la magenta absorbe verde, y la amarilla absorbe azul. Por ej, para ver verde en el papel, como parto de blanco, uso tintas cian y amarillas, para que absorban rojo y azul, y quede reflejado solo el verde.

Como en la impresoras la combinación de tintas no sale bien (ver slide anterior), el negro formado por la suma de los 3 colores suele quedar como una especie de gris o verde muy oscuro (y además el punto menos definido en el papel). También ocurre que el negro se usa mucho y así gastaría las 3 tintas, mientras que la tinta negra es más barata. Por todo esto es que las impresoras agregan un cartucho o tóner "negro". Es solo una cuestión técnica, que en el modelo teórico no sería necesario.

Entonces, se reemplaza el mínimo común de los 3 colores por "un poco" de negro para ahorrar y mejorar el resultado. Si un color por ejemplo tendría, $C=30\%$, $M=50\%$, $Y=20\%$; se puede reemplazar por $K=20\%$, $C=10\%$, $M=30\%$, $Y=0\%$ (saco 20% de cada color CMY y lo reemplazo por 20% de K).

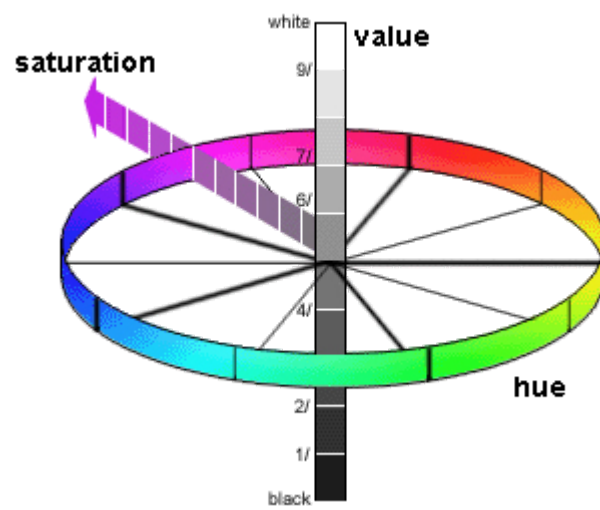
Las ecuaciones para pasar de un modelo a otro que vamos a usar son muy simples*:

- $R=1-C$, $G=1-M$, $B=1-Y$
- $C=1-R$, $M=1-G$, $Y=1-B$.

*las reales son más complejos porque no son lineales y la no-linealidad depende de cada dispositivo.

MODELOS HSL Y HSV

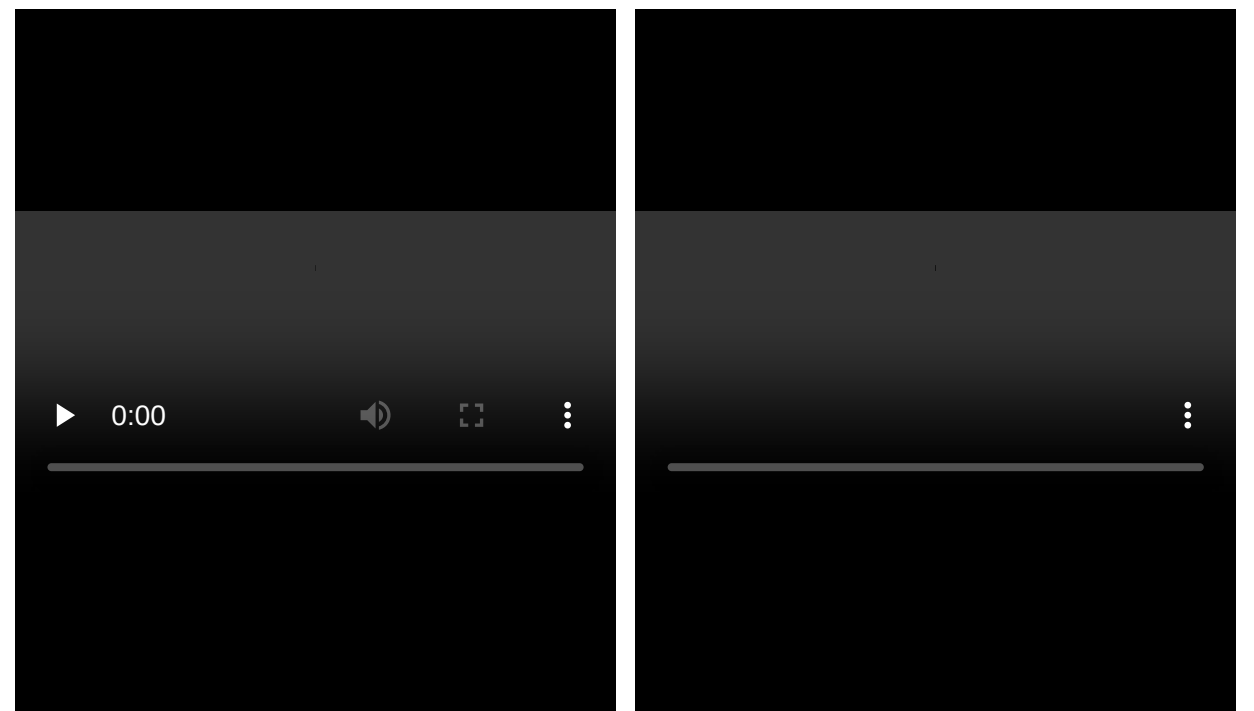
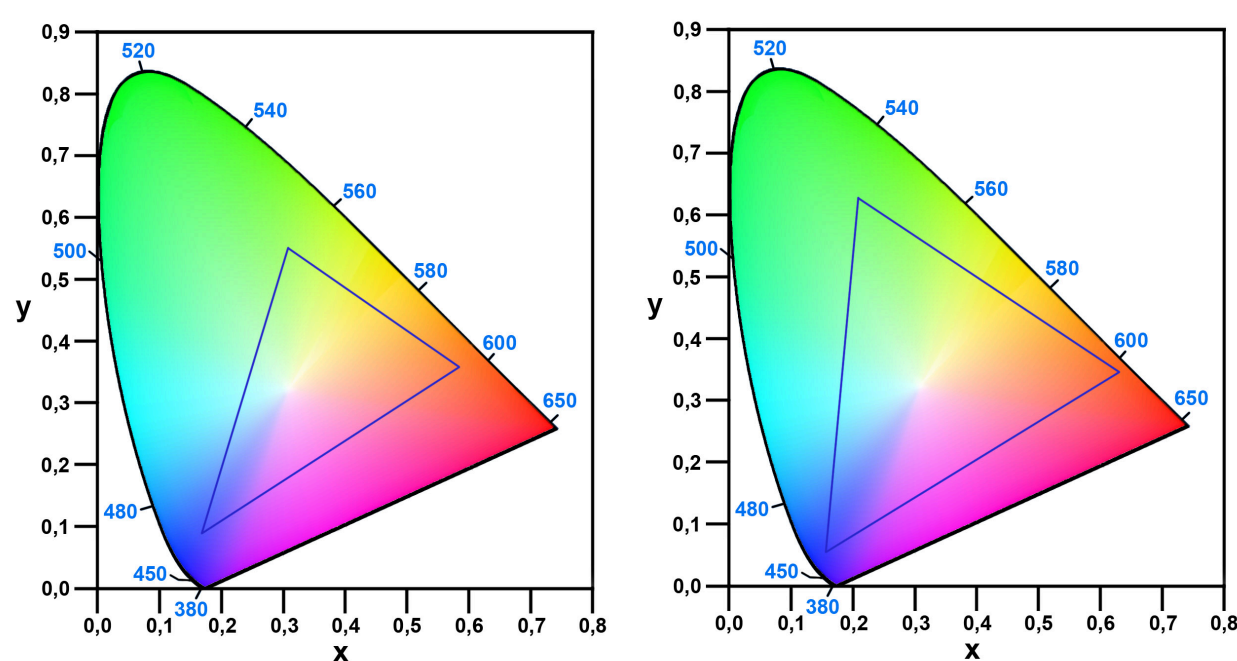
- Modelos cilíndricos, más "intuitivo" que el cubo RGB.
 - Hue (el ángulo) define el "tono".
 - Saturation (el radio) define cuán "puro" es.
 - en el eje están los "grises", hacia afuera los colores "vividitos".
 - Value (altura) define el brillo
 - arriba=blanco, abajo=negro.



Los modelos RGB y CMY son muy buenos, útiles, justificables técnicamente, pero poco intuitivos para una persona "común". Los modelos HS[Algo] (HSV, HSL, HSI, todos muy parecidos) buscan definir un color con 3 valores que sean más fáciles de explicar/entender. En una interfaz de usuario final, la definición de un color mediante estos modelos puede ser mucho más user-friendly que RGB o CMY si el usuario no es informático.

H (Hue o tono) hace referencia al tono, a lo que llamaríamos muy a la ligera "color" si no lo hubiéramos definido mejor antes. La dimensión S (saturación), define si es un color más vívido (S alto) o pastel/apagado (S bajo). Cuando S baja y el color deja de ser "puro", puede pasar a tender a blanco si V es alto, o a negro si V es bajo. Esta 3ra dimensión en estos modelos controla el "brillo", la "luminancia" o algún valor de interpretación similar.

DIAGRAMA CIE Y GAMUT



- La CIE define un "espacio" de color basado en un "observador estándar".
- Permite comparar dispositivos (monitores, impresoras, etc).

Los modelos que definimos hasta ahora no identifican un color en términos absolutos. R, G y B todos al máximo decimos que representa "blanco". Pero cuán blanco es ese blanco, depende de la calidad y configuración del monitor. Un monitor mejor dará un blanco más blanco/brillante que uno malo. Y aún un mismo monitor varía de acuerdo a su configuración de brillo (en modo ahorro de energía, el blanco ya no es tan blanco). Lo mismo para cualquier terna RGB: exactamente cómo se ve, depende del dispositivo.

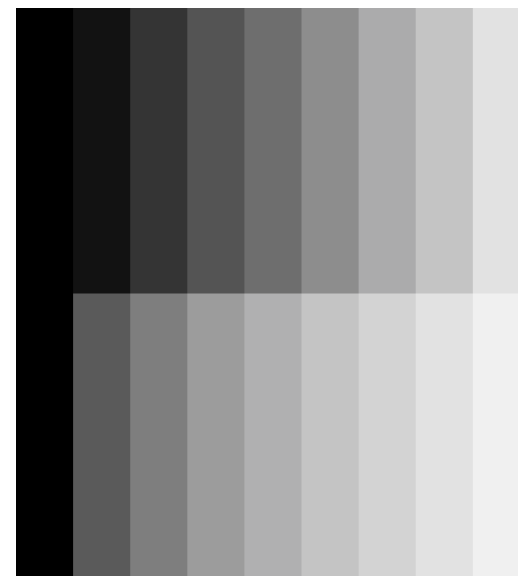
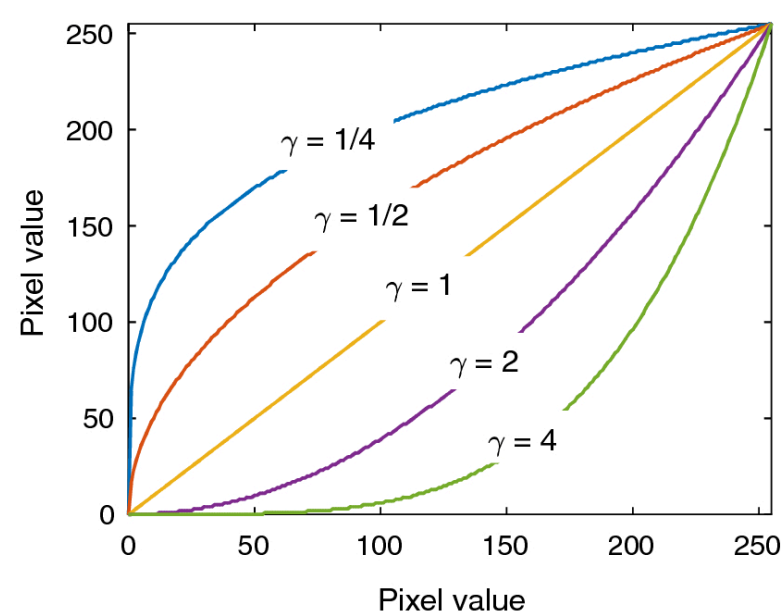
Los espacios de color CIE (hay varios CIE-algo) permiten definir un color también mediante 3 coordenadas (digamos XYZ, ya no son RGB), que toman como referencia al ojo humano promedio. Como resultado de experimentos con muchas personas se encontraron las capacidades de un ojo humano "estándar", y en base a eso se arma este espacio. Entonces, cuando queremos medir cuán blanco es el blanco del monitor, podemos ubicar su blanco en este espacio, y usarlo para comparar objetivamente con el blanco de otro dispositivo. Así surgen la idea de "gamut" y el "diagrama CIE".

Se le llama "gamut del dispositivo" al conjunto de colores que el dispositivo (monitor, cañón, impresora) es capaz de reproducir o representar. El diagrama CIE es una representación de este espacio donde se marca/pinta el área de los colores que puede ver el ojo humano, y ahí dentro se pueden señalar los que el dispositivo es capaz de reproducir. Típicamente, en la versión 2D (que muestra solo un corte, pues el espacio es 3D) el gamut será un triángulo o similar, donde los vértices coincidan con el verde, rojo y azul que usa en particular ese dispositivo. Cuanto mejores sean esos colores base (más alejados, o hacia el borde del diagrama estén), más colores podrá abarcar/reproducir.

Esta es entonces la herramienta para comparar más objetivamente la capacidad de dos dispositivos o tecnología diferentes. O incluso para justificar si agregar un cuarto led de un color X podría aportar algo a la capacidad del monitor (en términos de lograr más colores... pero hay otras cosas que pueden variar como por ejemplo el consumo de energía: un subpixel blanco tal vez ahorraría luz así como el tóner negro ahorra tinta en una impresora).

CORRECCIÓN GAMMA

- La respuesta del ojo no es lineal
- Tampoco de los sensores de las cámaras
- Ni de los leds del monitor



En los viejos monitores de tubo (CRT), la relación entre el voltaje aplicado al cañón de electrones y la intensidad de la luz emitida no era lineal. Que no sea lineal quiere decir que no es lo mismo aumentar 10% desde 10% a 20%, que desde 80% a 90%. O que 50% de la potencia total no es 50% de la intensidad de luz máxima. Para corregir esta relación se le aplicaba a la entrada una "corrección gamma". Si imaginamos cada entrada (los canales R, G y B) como un valor entre 0 y 1, podemos alterar ese valor elevándolo a una potencia. Al exponente se lo denomina con la letra gamma, y por a esto se le llama función "corrección" gamma.

Por ejemplo, con gamma=2 tenemos que $0.5^2=0.25$. Esta relación no lineal convierte 1/2 en un 1/4. A esto me refería con que el 50% no es el 50%.

Pero también sucede que la percepción del ojo tampoco es lineal. Somos más sensibles a variaciones relativas en colores oscuros, y menos a las mismas variaciones cuando ocurren en colores claros. Y el sentido de esta no linealidad es casi el opuesto al de los viejos monitores CRT. Entonces, se compensaban (quiero mostrar un valor, el monitor por su no linealidad produce otro, pero el ojo por su no linealidad lo percibe diferente en el sentido contrario, y entonces queda casi como el original que quería mostrar).

Esto sirve también para aprovechar mejor la codificación del valor de intensidad. Si codificamos cada canal con 8bits por ejemplo, en cada canal podemos representar $2^8=256$ niveles de intensidad diferentes. Serían un desperdicio que estén equiespaciados; mejor que "amontonemos" más niveles donde el ojo es más sensible.

Por esto y la compatibilidad hacia atrás, los monitores actuales podrían dar una respuesta lineal, pero emulan el factor gamma de los viejos CRT.

El problema es que si queremos calcular luego iluminación según las leyes de la física, cuando una ecuación diga que un objeto refleja la mitad de la intensidad... ¿será la mitad real? ¿o la mitad en esa curva corregida con un factor gamma? Hay que tener en cuenta esas correcciones. Si el dato de entrada no es lineal, hay que aplicar un gamma inverso al que tenga aplicado de origen para hacerlo lineal otra vez, ahí operar, y luego aplicar el gamma necesario para la no-linealidad del monitor.

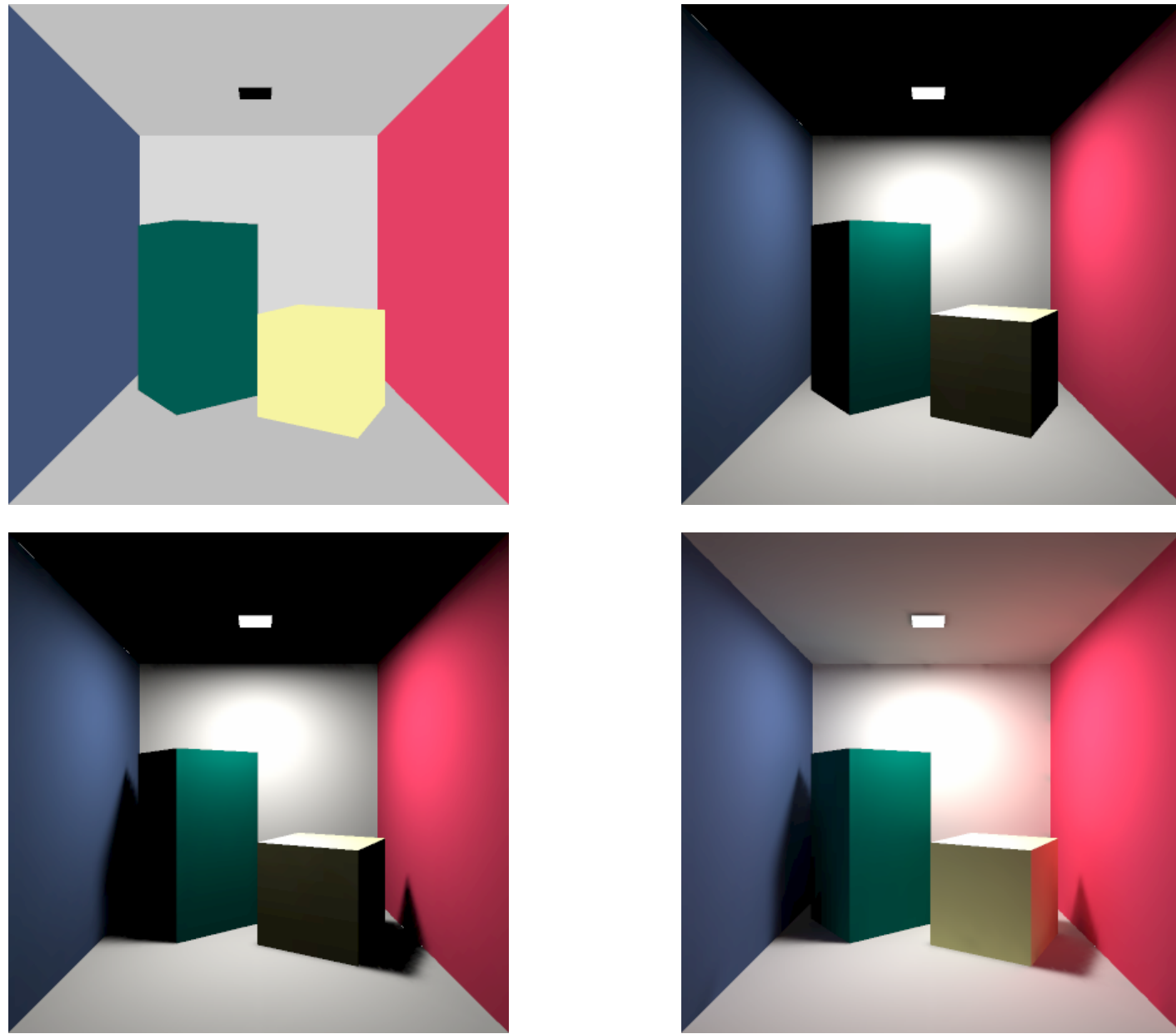
Cuando queremos implementar algo físicamente correcto, esta clase de cosas se tornan detalles importantes. En nuestro caso, dado que nuestro modelo ya será bastante poco realista, no nos tomaremos la molestia de tener esto en cuenta en las implementaciones. Pero un modelo que aspire a ser físicamente correcto debería hacerlo.

Parte 2:

Iluminación

Dijimos en la clase de introducción que un algoritmo de rendering tiene 2 grandes problemas que resolver. Determinar qué se ve (lo cual podía hacerse pixel por pixel tirando rayos, o primitiva por primitiva rasterizándolas), y cómo se ve lo que se ve. Lo segundo implica calcular iluminación: determinar cómo interactúa ese objeto (ese punto de la superficie en realidad) con la luz para saber de qué color exacto lo veremos.

COLOR VS. ILUMINACIÓN LOCAL VS. GLOBAL



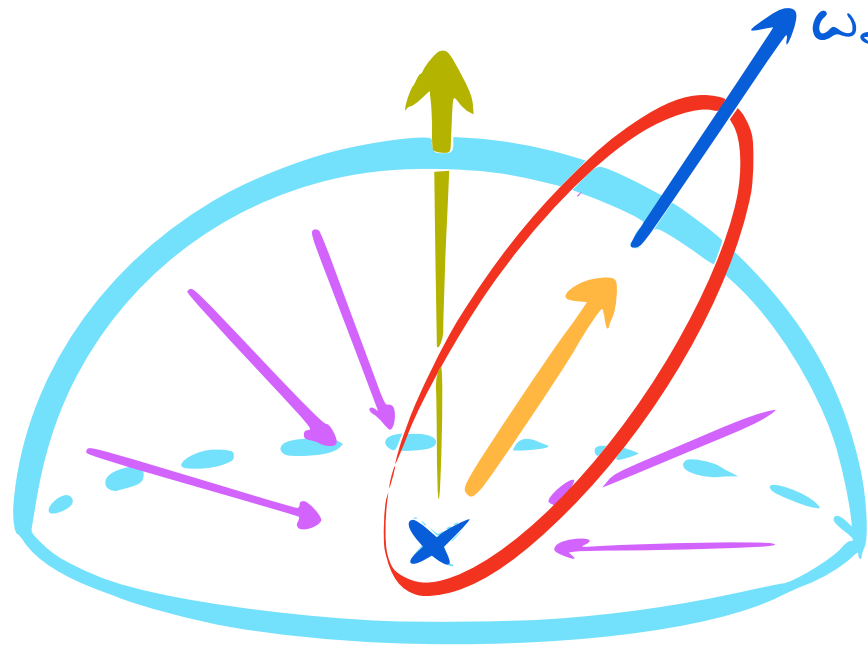
La primera imagen no calcula iluminación, sino que simplemente pinta cada objeto con su color asignado sin alteraciones. Pintando de esta forma, las imágenes generadas parecerían más 2D que 3D (por ej, sería imposible distinguir una esfera de un círculo plano).

La segunda, calcula el sombreado de cada objeto. Esto es, tiene en cuenta en cada punto de la superficie, la interacción con la luz que recibiría directamente desde la fuente de luz, para determinar qué partes se verán más iluminadas y cuales más oscuras. Esto alcanza para transmitir la sensación de 3D/profundidad de la escena, pero no es del todo realista. Podemos ver, por ejemplo, que los cubos no proyectan sombra en la pared (notar que sombreado!=sombra).

A esto, cuando para "sombrear" un objeto solo consideramos la iluminación directa desde la fuente e ignoramos al resto de los objetos, le llamaremos iluminación local. Es lo que se implementa fácil y eficientemente en un pipeline de renderizado por rasterización, ya que el pipeline procesa las primitivas individual e independientemente. No hay forma fácil de calcular la sombra que proyecta un objeto sobre otro si vemos solo una primitiva por vez (más aún, esto se calcula por fragmento, no por primitiva).

En la tercera se agregan las sombras. Esto suma algo de realismo a la imagen, pero no es lo único que nos estábamos perdiendo en la segunda. La última imagen considera también la iluminación indirecta. Es decir, la luz que llega a un objeto luego de haber rebotado en otro, y no directamente desde la fuente. Esta diferencia se nota claramente en el techo, o en las caras de los cubos que antes estaban a oscuras. Este último caso corresponde verdaderamente a iluminación global, donde para sombrear un objeto (o una primitiva) se tienen en cuenta las demás y las contribuciones de luz indirectas. Esto es obviamente más caro y más realista. En general se logra con técnicas basadas en raytracing.

LA ECUACIÓN DEL RENDERING



$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \underbrace{\int_{\Omega} L_i(\mathbf{x}, \omega_i) \cdot f_r(\mathbf{x}, \omega_i \rightarrow \omega_o) \cdot \cos\theta_i d\omega_i}_{L_r(\mathbf{x}, \omega_o)}$$

Esta es la ecuación que vamos a querer modelar y resolver para calcular iluminación. Describe cómo rebota la luz en un punto.

$L_o(\mathbf{x}, \omega_o)$ es la luz que rebota/sale del punto $\mathbf{x}$ en la dirección ω_o .

A la izquierda hay un término para representar la luz que el objeto emite en esa dirección (L_e) y una integral que representa la luz que recibe de otras fuentes y rebota.

La integral es para considerar todas las "infinitas" direcciones desde las cuales puede llegar luz (por eso una semi-esfera alrededor de la normal).

$L_i(\mathbf{x}, \omega_i)$ representa la luz que llega al punto $\mathbf{x}$ desde una dirección particular ω_i .

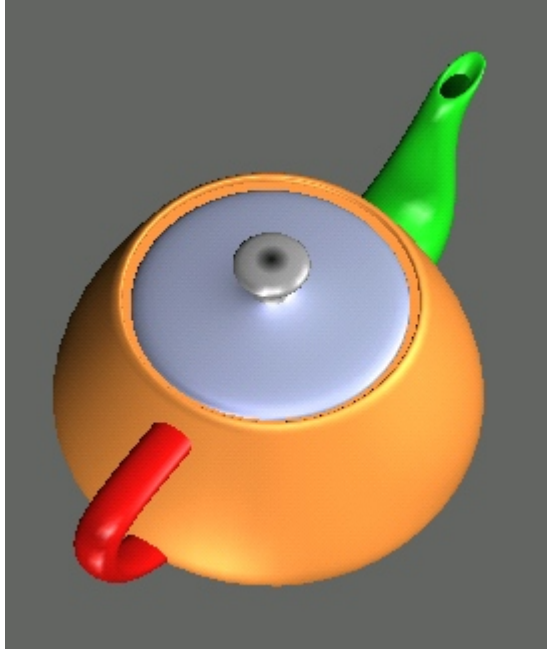
$f_r(\dots)$ es una función que define qué proporción de lo que llega rebota, y es la función importante a modelar, porque allí dentro se codifican las propiedades del material (que no es solo el color del objeto, sino también, por ej, si es un material pulido y brillante o rugoso/mate, metálico o no, etc). En lo que sigue de la clase, veremos un modelo local, y estaremos hablando mayormente de cómo aproximamos f_r .

$\cos(\theta)$ es un término "geométrico" que corrige la cantidad de luz por unidad de área según el ángulo de incidencia.

En un modelo local, se consideran solo las direcciones hacia las fuentes de luz, y son estas fuentes las que determinan el término L_i . En un modelo global el cálculo de L_i es mucho más complejo. Si tenemos implementada la "maquinaria" necesaria para tirar rayos y encontrar intersecciones eficientemente, una forma de aproximar esta ecuación es tirar rayos para "muchas" direcciones de la semi-esfera (no solo hacia las luces) con el objetivo de aproximar mejor esa integral (muestrear mejor la luz que incide) y ver por cada rayo a qué le pega. Si le pega a una fuente de luz, será una contribución directa. Si le pega a otro objeto, podemos aplicar la misma ecuación recursivamente en ese otro objeto para saber cuánta luz habría rebotado desde el mismo. Entonces, en raytracing tenemos una forma muy simple de calcular iluminación global, aunque sin optimizaciones esta forma es muy costosa (la cantidad de rayos crece exponencialmente).

ILUMINACIÓN LOCAL VS. GLOBAL

Local



- ▶ Solo tiene el objeto, la luz y el ojo (iluminación directa).
- ▶ Rasterización+Phong/PBR

Global

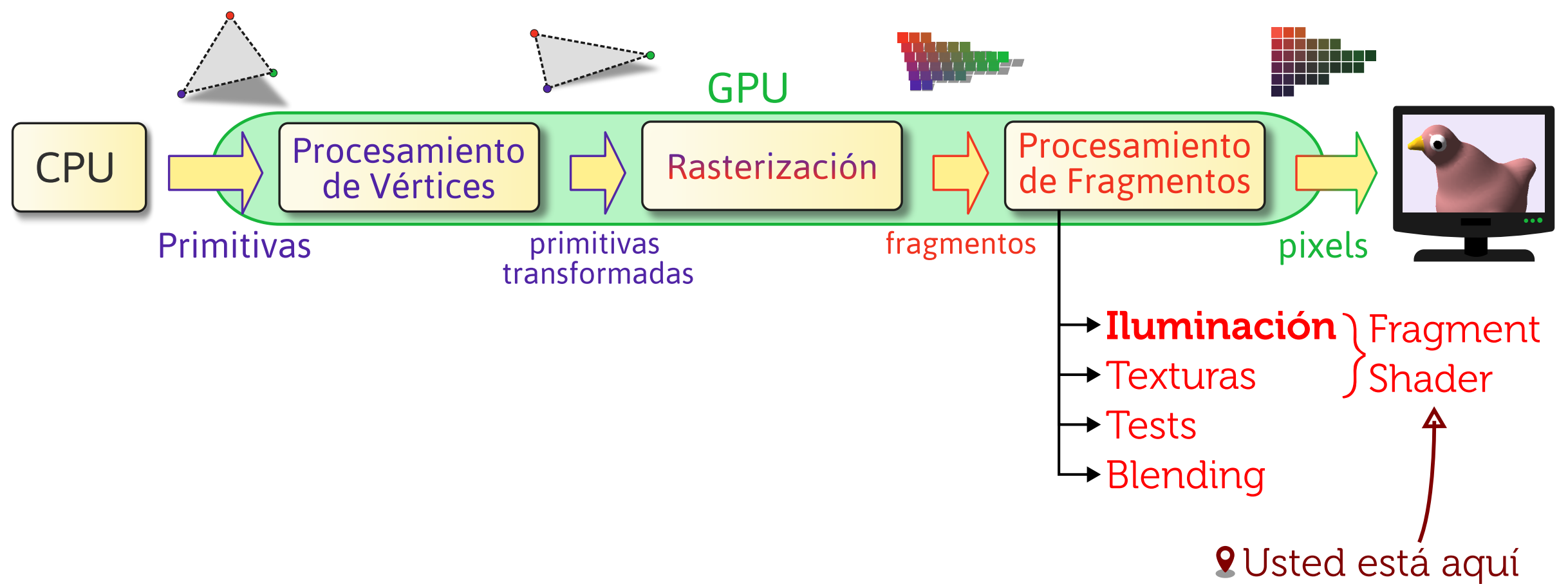


- ▶ Considera también a los demás objetos (iluminación indirecta).
- ▶ Raytracing, Radiosity, Photon Map, etc

Por todo lo dicho, los modelos locales obtendrán resultados de mucho menor calidad que los globales. Pero serán mucho más rápidos. En general, en tiempo real se usan modelos locales y se tapan los defectos simulando parte de lo que el modelo se pierde (como sombras y reflejos) con modificaciones ad-hoc (y muchas veces caras y complejas) al algoritmo de rendering.

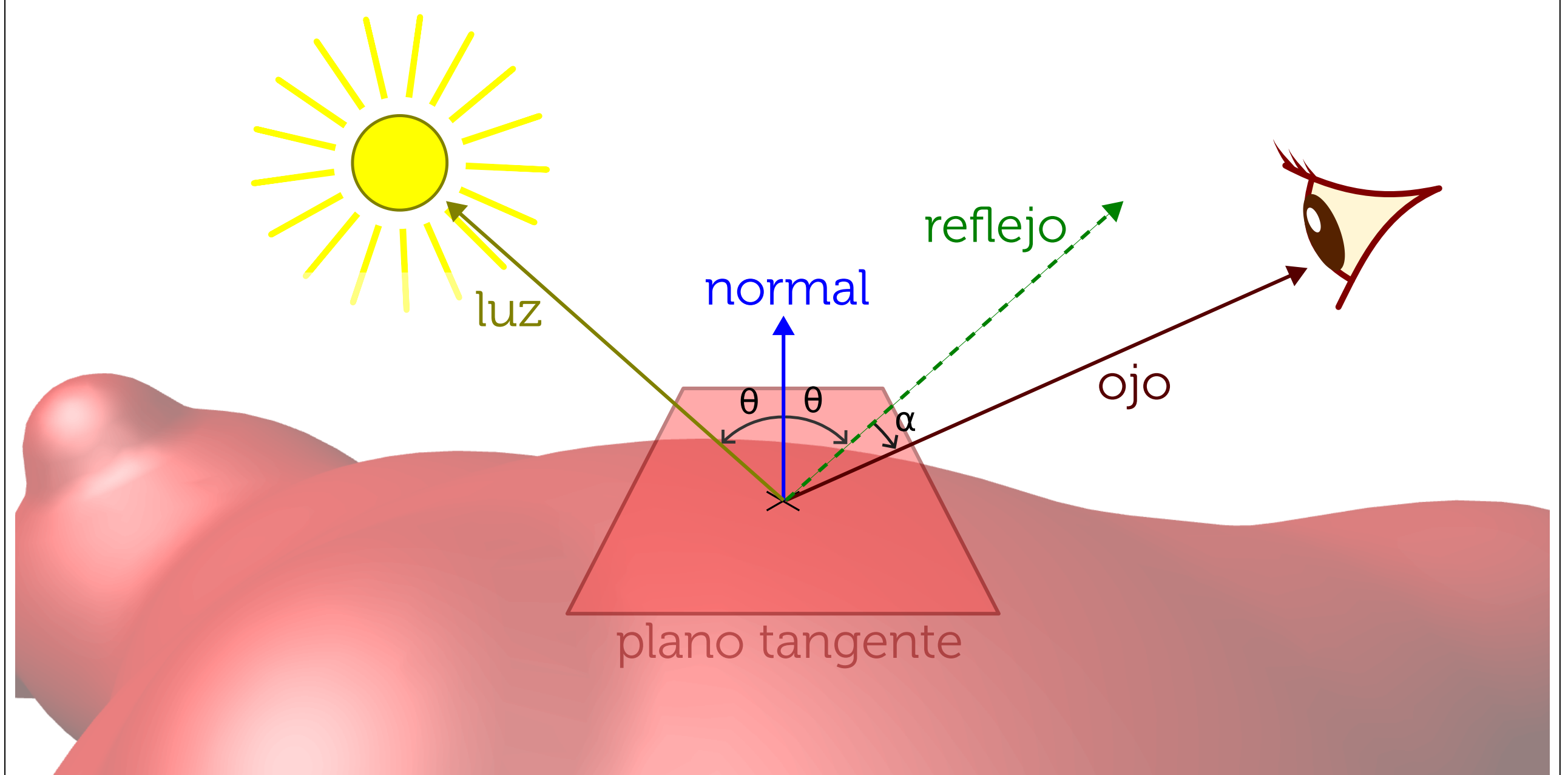
En la materia, nos concentraremos en mayor medida en lo básico (que lamentablemente es el caso de la izquierda), pero llegaremos a mencionar algunos de los trucos para suplir las limitaciones, y hasta implementaremos "sombras" en un TP.

ILUMINACIÓN EN EL PIPELINE



Dentro del pipeline, el momento de determinar el color de un píxel es durante el procesamiento de fragmentos. Por cada fragmento, se ejecuta el fragment-shader, que es el programa responsable de esto, y que nosotros debemos escribir. Allí usualmente se calcula iluminación, y se aplican texturas. Luego, el fragmento pasa por los tests (ej: el algoritmo del z-buffer para resolver las oclusiones) y finalmente si sobrevive se lo pinta. Aquí, dependiendo del canal alpha que mencionamos en la parte de Color, el color del fragmento irá directo a la imagen, o se mezclará (blending) con lo que había previamente para dar una sensación de transparencia.

BIDIRECTIONAL REFLECTANCE DISTRIBUTION FUNCTION



A las funciones que definimos para calcular qué proporción de la luz que llega a un punto rebota en cierta dirección se las suele llamar BRDF. Lo de bi-direccional (BD) es porque solo tienen en cuenta dos direcciones importantes: de dónde viene la luz, y hacia dónde quiero calcular el rebote (usualmente, la dirección hacia el ojo es la que nos importa).

Para hacer esta aproximación, supondremos que la superficie es suave en el punto donde estamos calculando (que hay un plano tangente), y tomaremos como dato representativo la normal en ese punto.

NORMALES, FRAGMENTOS Y PRIMITIVAS



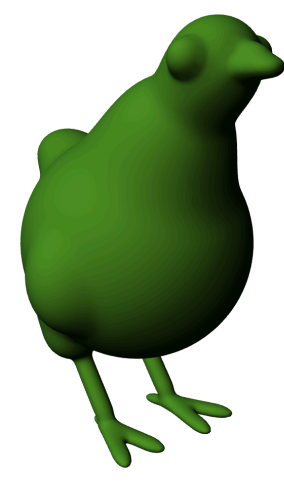
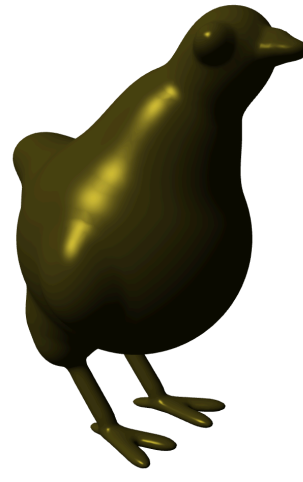
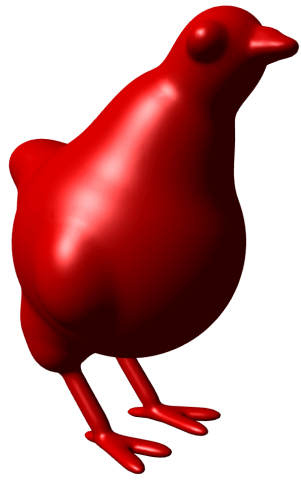
Notar que en el caso de rasterización, el "punto" sobre la superficie se corresponderá con el centro de un fragmento. Pero la normal en ese punto no será la normal de la primitiva que lo generó. Si usáramos para iluminación las normales de las primitivas, acentuaríamos en la imagen el hecho de que los objetos estén aproximados por primitivas, porque la normal variaría bruscamente al pasar de una primitiva a otra, haciendo visible la arista que las une.

Lo que realmente se hace es asociar una normal a cada vértice (que será la normal de la superficie real/original), e "interpolarse" durante la rasterización una normal para cada fragmento. De esta forma, el cambio de normal entre primitivas es suave, porque si dos triángulos comparten una arista, ahora comparten las normales de los vértices que definen esa arista, y el mecanismo de interpolación generará normales por fragmentos muy similares a ambos lados la misma.

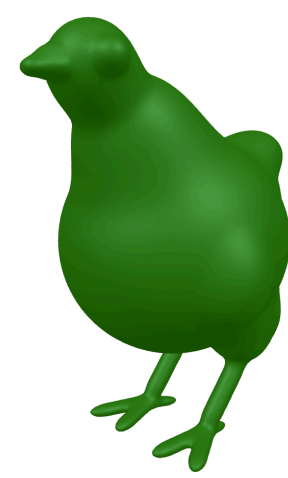
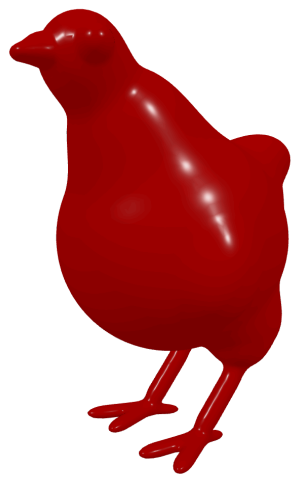
En esta slide, ambas imágenes utilizan exactamente las mismas primitivas. A la izquierda, se calcula iluminación utilizando las normales de las primitivas (se le suele llamar sombreado "plano" o "flat"). A la derecha, cada vértice tiene asociada una normal (dato) y la iluminación se calcula usando normales interpoladas para los fragmentos.

MODELOS DE ILUMINACIÓN/SOMBREADO

- Fenomenológicos: Phong y Blinn-Phong



- Físicos: PBR (Physical Based Rendering)

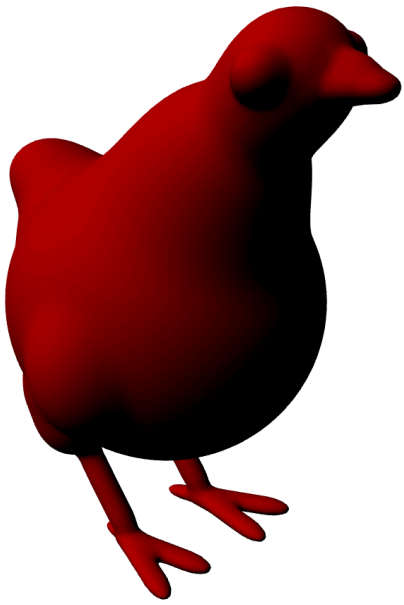


Los modelos "fenomenológicos" se basan en observaciones. Phong buscó ecuaciones simples y eficientes que aproximaran ciertas observaciones que hizo sobre cómo se ven los objetos bajo distintas condiciones de iluminación. El modelo alcanza para dar una sensación de profundidad/3D, e incluso distinguir distintos materiales (más o menos pulidos), aunque no reproduce de forma realista o convincente a la mayoría de ellos (el resultado casi siempre se parece a una especie de plástico). Estos modelos en general tienen pocos parámetros y que deben ajustarse "a ojo" para definir un material particular. Al no ser basados en física, un conjunto inadecuado de parámetros hasta puede dar lugar a cosas muy poco creíbles, como que refleje más luz de la que recibe ("Homero! En esta materia no respetamos las leyes de la termodinámica").

Los modelos basados en física se basan en leyes de la física. Analizando cómo interacciona efectivamente la luz con el material según leyes conocidas (que usualmente refieren a características microscópicas), se plantean ecuaciones que resuman estadísticamente ese comportamiento. Estos modelos serán en general más complejos, tanto para la implementación, como para la definición del material. Lo segundo implica más parámetros que definir para dar el aspecto de cierto material. Pero si están basados en física, muchas veces los parámetros se pueden "medir" o estimar de la realidad (con "fotos" de un objeto real) de alguna forma.

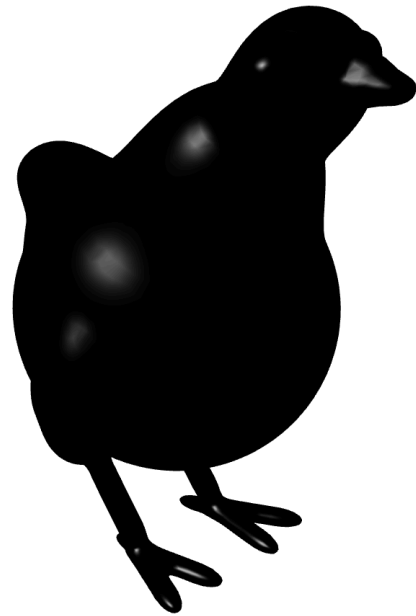
EL MODELO DE PHONG

Difusa:



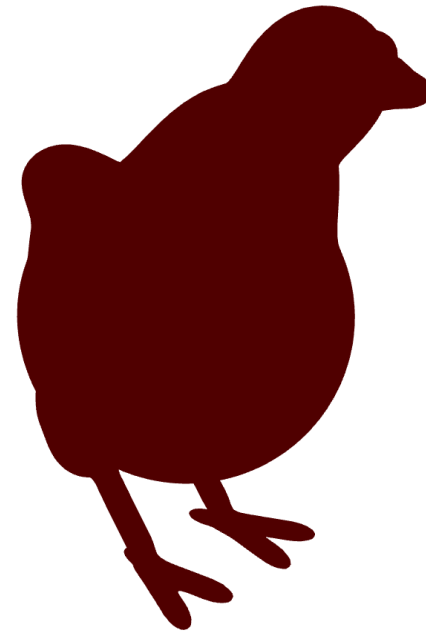
- Color base
- Superficie mate o Lambertiana

Especular:



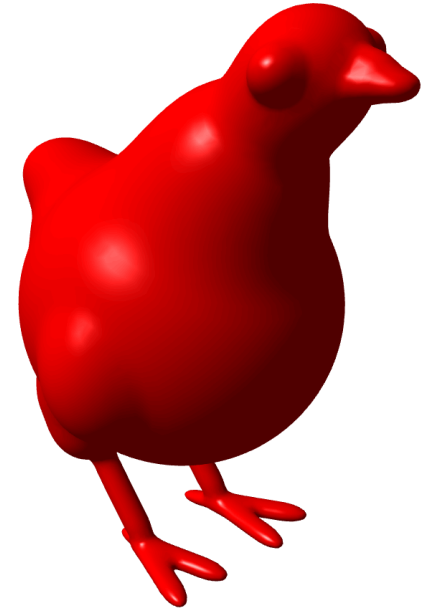
- Brillo en objetos pulidos
- Espejo no ideal

Ambiente:



- Iluminación indirecta

Resultado:



- Suma (clampeada!)

El modelo de Phong es un modelo fenomenológico que propone calcular la luz reflejada como la suma de 3 componentes:

La componente difusa es la que da el color base y la sensación de profundidad. Modela relativamente bien materiales mate, rugosos, o poco pulidos/brillosos (tela, tiza, tierra).

Para agregar el detalle del brillo en materiales más pulidos (plástico, metal) se añade la componente especular.

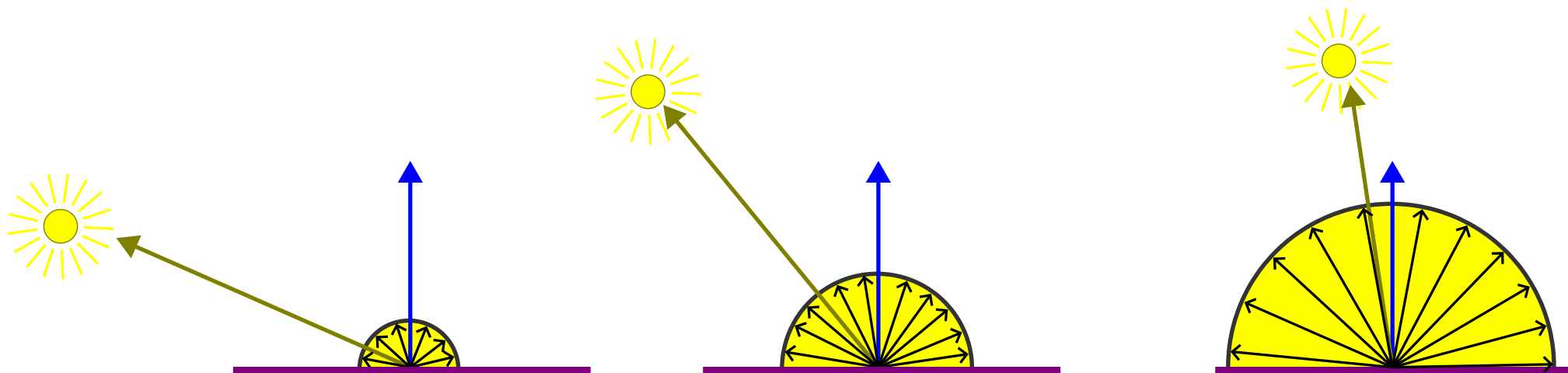
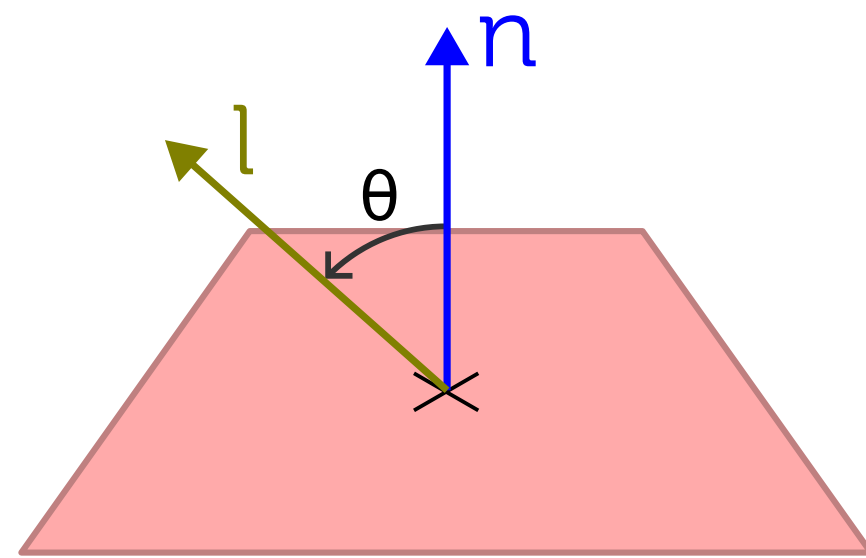
Finalmente, como es un modelo "local", se "pierde" todas las contribuciones indirectas, que son las que en la realidad hace que casi ningún punto se vea realmente negro. Para evitar que esto pase en la escena renderizada, se agrega a la iluminación una cantidad de luz arbitraria y fija, la componente ambiente, que busca compensar esa iluminación indirecta no contemplada.

El resultado es la suma (clampeada, porque podría dar más de 1) de las 3 componentes.

EL MODELO DE PHONG: COMPS. AMBIENTE Y DIFUSA

$$I_a = K_a I$$

$$I_d = K_d I \underbrace{\cos(\theta)}_{n \cdot l}$$



En todas las ecuaciones de estas slides sobre el modelo de Phong debemos tener en cuenta que:

- cada ecuación en realidad se aplica 3 veces, una para saber cuánto refleja de rojo, otra de verde y otra de azul.
- en cada ecuación, las constantes I_{algo} corresponde a una configuración de la fuente de luz, mientras que las K_{algo} corresponden al material del objeto.

La componente de ambiente simplificará todas las contribuciones indirectas sumando simplemente un valor constante, que solo dependerá del color y la intensidad de la fuente de luz. Dado que hablamos de contribuciones "indirectas", sus valores deberían ser relativamente bajos. Para ello se suele colocar en la constante de la luz una intensidad baja (10% o 20% del color de esa luz), y en la constante K el color real del objeto (que se verá entonces "oscurecido" por el producto con esa I).

La idea detrás de esta componente es simular una superficie mate o Lambertiana ideal. En este caso, cuando la luz choca contra el objeto, se distribuye por igual en todas las direcciones. Entonces el resultado de esta ecuación no depende de la dirección de la vista. Pero sí depende de cómo le llega la luz: será más intensa cuando la fuente de luz lo ilumine perpendicularmente, y se irá "apagando" hasta que sea 0 a partir de los 90 grados.

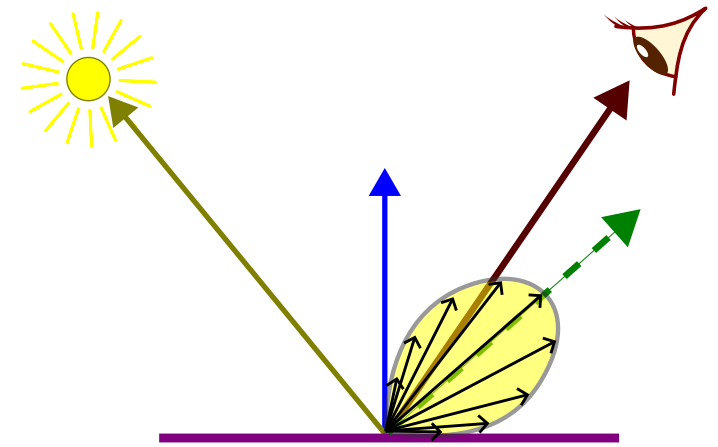
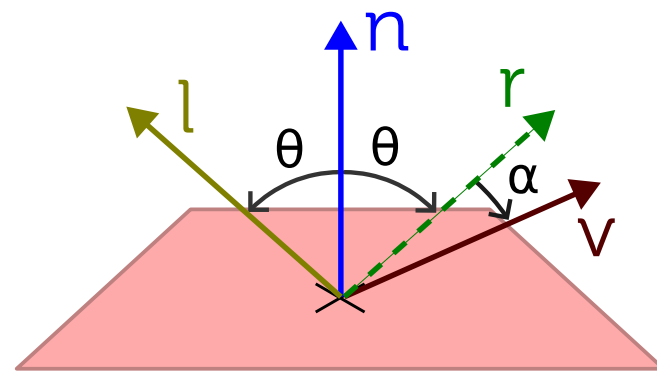
En estas ecuaciones usaremos un coseno para modelar ese decaimiento respecto del ángulo, porque es una función suave, porque es barato de calcular (si los vectores normal y hacia la luz están previamente normalizados, el producto punto da el coseno), y porque a veces hasta resulta físicamente correcto (si intentan pensar en un rayo de luz que sea un volumen/cilindro con cierto radio y no solo una "línea", la relación entre el área donde impacta en la superficie, y el área de un "corte" perpendicular de ese rayo está dada por el coseno).

En las imágenes de abajo, la dirección de la luz varía, la normal permanece fija. El semi-círculo amarillo representa cuánto refleja en cada dirección (dada una dirección, el "radio" en esa dirección es proporcional a la cantidad de luz reflejada).

EL MODELO DE PHONG: COMPONENTE ESPECULAR

Phong original:

$$I_s = K_s I \underbrace{\cos^q(\alpha)}_{(r \cdot v)^q}$$



La componente especular modela un reflejo especular no ideal.

- En un espejo ideal, una fuente de luz puntual se ve en un solo punto, aquel donde el rayo hacia la vista coincide con el rayo de la luz reflejado respecto a la normal. Si la vista se mueve, ya no verá el reflejo de la luz en ese punto.
- En un espejo no ideal (la superficie no está perfectamente pulida), al mover la vista de la dirección del rayo reflejado, todavía podemos recibir algo de luz. No "toda" luz rebota en la dirección del rayo reflejado, sino mayormente en esa dirección, pero se distribuye con menor fuerza en las direcciones similares (alrededor de el rayo reflejado).

(pregunta de álgebra para un examen: ¿cómo calculo el rayo reflejado? tener en cuenta que estamos en 3D)

Modelamos el decaimiento del reflejo a medida que nos alejamos del rayo reflejado nuevamente con un coseno, pero esta vez elevado a una potencia q . Aumentar q hace que la intensidad disminuya más rápidamente cuando la vista se aleja del rayo reflejado. Un espejo ideal correspondería a un q infinito. Una superficie muy brillante/bien pulida tendrá un q alto (¿qué es alto? depende de la implementación, pero digamos que 128 o 256 son máximos habituales), mientras que una poco brillante un q más bajo (más cercano a 1).

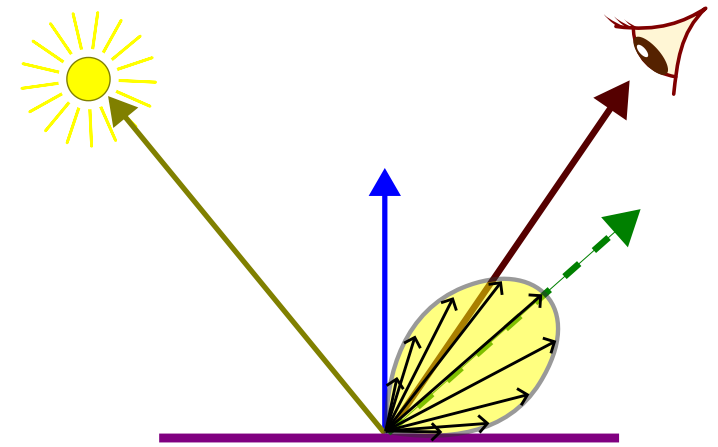
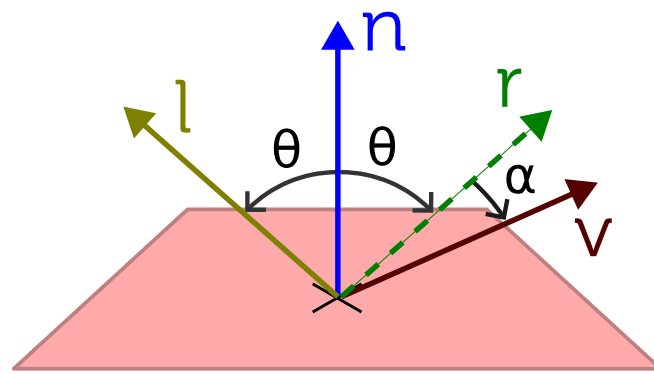
Sin embargo hay que notar que si solo disminuimos la q , estamos haciendo que refleje más luz (el coseno igual vale 1 en R, pero decae más lento alrededor, se ve un área de brillo mayor). Si quiero pensar en algo parecido a la conservación de la energía (para que los parámetros seleccionados tengan algún sentido), al bajar q también debería bajar K_s , de forma que la integral de $K_s \cdot \cos^q$ (que sería la cantidad total de luz reflejada) se mantenga.

El valor q depende entonces del material (como las K) y no del objeto. Y el color de K_s será habitualmente blanco, ya que en la mayoría de los materiales el brillo se ve blanco aunque el material sea de otro color.

EL MODELO DE PHONG: COMPONENTE ESPECULAR

Phong original:

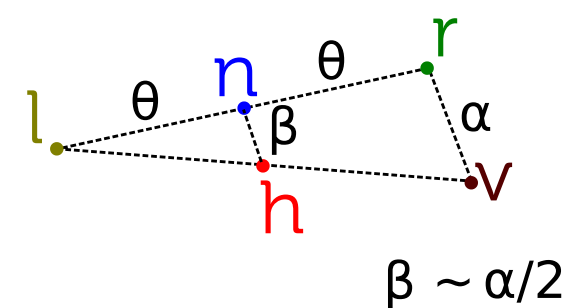
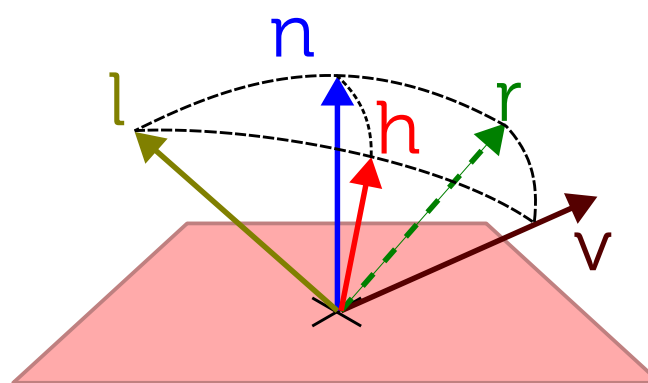
$$I_s = K_s I \underbrace{\cos^q(\alpha)}_{(r \cdot v)^q}$$



Blinn-Phong:

$$I_s = K_s I \underbrace{\cos^q(\beta)}_{(n \cdot h)^q}$$

$$h = (l + v) / |l + v|$$



Por último, hay una variación propuesta por Blinn que es la que se utiliza habitualmente en OpenGL. Se basa en observar que los vectores son normales (todos módulo 1, todos apuntan a la superficie de una esfera imaginaria de radio 1 y centrada en el punto a iluminar), y que forman dos pseudo-triángulos semejantes (son triángulos sobre la esfera, sus lados entonces son curvos). Uno con vértices en L (dirección hacia la luz), R (dirección del rayo reflejado) y V (dirección de la vista); y otro con vértices en L, N (normal de la sup.) y H (un vector nuevo a calcular). Como N está a medio camino entre L y R, calculamos un H a medio camino entre L y V, y así los triángulos son semejantes, teniendo el segundo longitudes de lados que son la mitad de las del primero. Entonces, el ángulo alfa que nos interesaba, se puede obtener como $2 \cdot \beta$, que es más simple de calcular. Esto es en realidad una aproximación (sería exacto en un triángulo plano, no en uno esférico).

Se puede argumentar (no nos detenemos ahora en por qué) que el "error" de esta aproximación juega a favor de Blinn (su modelo se parece más a la realidad en ciertos casos). Y el coeficiente 2 se puede ignorar y compensar reajustando la q. Así llegamos a la versión que habitualmente se implementa de la componente especular.

EL MODELO DE PHONG COMPLETO

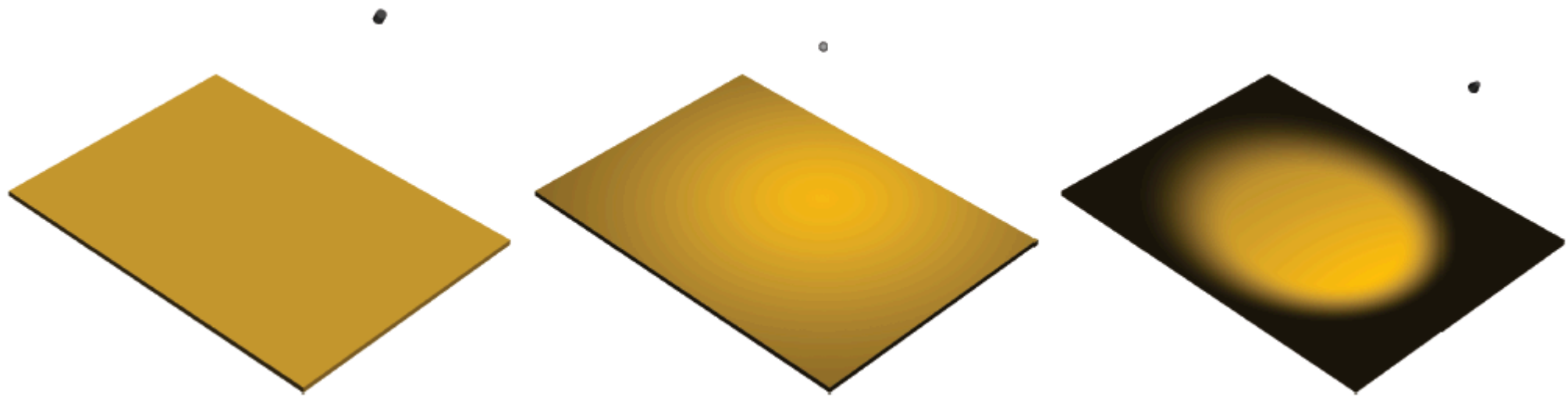
por cada fuente de luz....

$$I = \underbrace{K_a I_{ag}}_{\substack{\text{ambiente} \\ \text{global}}} + \underbrace{K_e}_{\text{emisiva}} + \sum_{j=0}^n \left(\underbrace{K_a I_{aj}}_{\text{ambiente}} + \underbrace{K_d I_{dj} \cos(\theta)}_{\text{difusa}} + \underbrace{K_s I_{sj} \cos^q(\beta)}_{\text{especular}} \right)$$

Llegamos finalmente a la versión completa que se implementa usualmente en OpenGL. Las componentes del modelo de Phong están dentro de una sumatoria (que hace las veces de la integral de la ecuación del rendering) para considerar múltiples luces y no solo una. Además de esto podemos agregar un término de ambiente global (para que no quede todo 100% negro cuando no hay ninguna luz "prendida"), y un término "emisivo" donde podemos modelar materiales que se ven aunque no reciban luz (un led, una llama, una luciérnaga).

Esta era la ecuación implementada en OpenGL cuando se tenía un pipeline de funcionalidad fija. El usuario solo modificaba los coeficientes (I_* , K_* , q). Ahora es responsabilidad del programador implementar el modelo de iluminación desde cero. Si no requerimos mayor realismo (si no vamos a PBR), esta ecuación será generalmente el punto de partida; y es lo que verán implementado en los fragment-shaders de todos los TPs.

EL RESTO DEL CAMINO



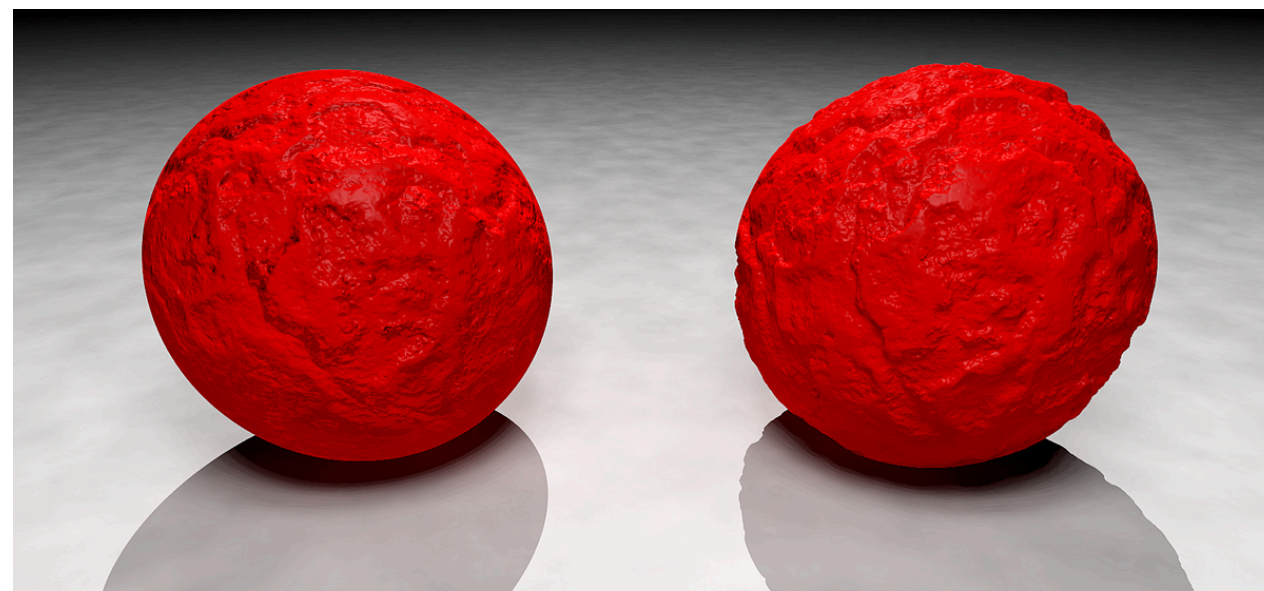
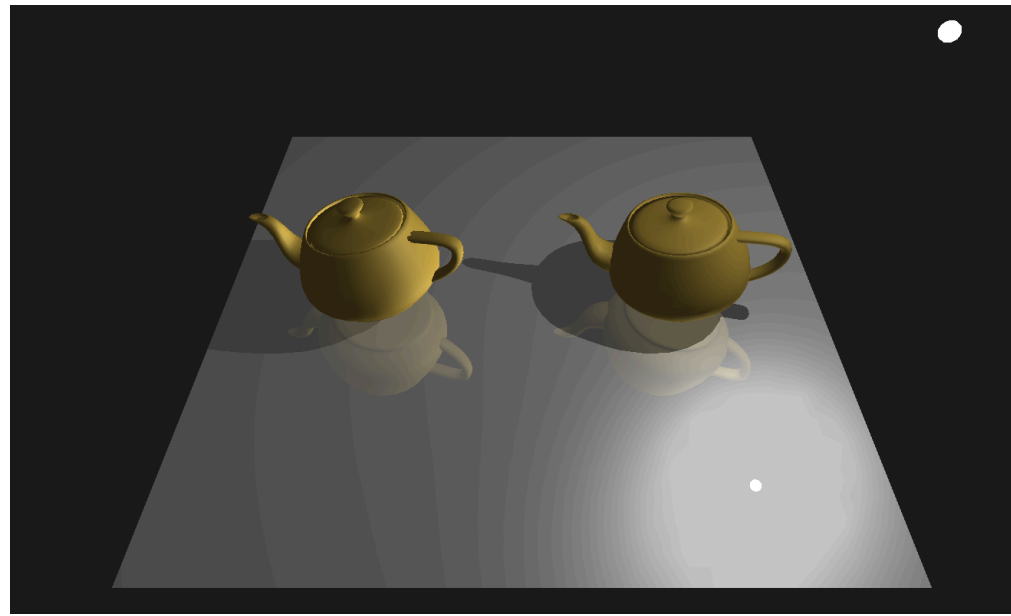
^ Atenuación con la distancia

< Niebla

La ecuación de Phong modela en realidad que pasa en el punto donde rebota la luz. Pero ¿qué le pasó a la luz que llegó, en el camino de la fuente a ese punto? y ¿qué le pasará a la luz que rebote, en el camino hacia el ojo? En una versión simplificada y poco realista, ignoramos estas preguntas y colocamos en las I del modelo de Phong las I originales de cada luz, y usamos directamente el color reflejado para pintar el fragmento.

Pero podríamos querer modelar, por ejemplo, el decaimiento de la luz con la distancia, o la mezcla del color reflejado con el color de una "niebla" presente en el ambiente. O una dirección preferencial para la fuente de luz (como en una linterna, que no ilumina para todos lados). Hay ecuaciones simples para esto que estaban implementadas en el pipeline de funcionalidad fija. Dado que ahora podemos implementar cualquier cosa, no nos detendremos en ellas (igualmente deberían ser fáciles de deducir), pero sí es importante saber que aquí hay dos simplificaciones que se podrían mejorar para lograr ciertos efectos.

MÁS...



A partir de las ideas básicas de iluminación se pueden hacer luego muchos trucos.

En particular suele ser interesante tratar de "agregar" sombras y reflejos. En la unidad de buffers veremos algunos trucos para lograr esto. Por ejemplo, la escena de las teteras no renderiza 2 teteras calculando realmente sombras y reflejos en el plano, sino que renderiza 6 teteras: las dos originales, 2 invertidas respecto al plano y con algo de transparencia para que parezcan reflejos, y dos "aplastadas" contra el plano para que parezcan sombras. Esto es relativamente simple de hacer/entender con un plano, pero no aplica a otra geometría. No es la única forma de lograr esto, pero si tal vez la más simple. En general cada "truco" que hacemos para compensar un defecto del modelo tiene sus pros y sus contras.

En la foto de las 2 esferas, una (derecha) realmente tiene a nivel geométrico el detalle del relieve (hay muchísimos triangulitos para copiar esas irregularidades), la otra (izquierda) es una simple esfera suave con un truco de iluminación (el contorno la delata). El truco consiste en alterar las normales. Se procesa un triángulo de una esfera "lisa", pero al momento de calcular la iluminación, se reemplazan las normales por las que habría tenido la superficie con "relieve", y entonces el efecto de la iluminación hace parecer que hay un relieve. Obviamente es más barato esto último, aunque requiere trabajo extra en el shader (y datos de entrada extra también).

¿QUÉ HAY QUE SABER?

- Color y Ojo
 - ¿Qué es un color? ¿Por qué RGB?
 - Diagrama CIE - Gamut
 - Modelos RGB, CMY[K], HSV
- Iluminación:
 - Local vs. Global
 - Fenomenológico vs Físico
 - **Modelo de Phong** completo
 - Saber deducir parámetros
 - Saber predecir resultados